

Nguyễn Quốc Bảo

Sinh viên Công nghệ kĩ thuật, ngành cơ điện tử, khóa K24

Điều khiển động cơ bằng PWM

Nhiệm vụ số 1:

_Điều khiển được động cơ bằng PWM

_Sử dụng được encoder + Điều khiển tốc độ động cơ + số chân cắm vào phải tương ứng với mạch

Nhiệm vụ nhỏ:

Xác định toàn bộ chân động cơ ở trong ảnh -> viết lại dưới dạng mạch breadboard ở fritzing

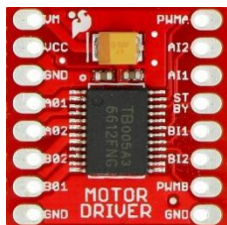
#MINI TASK: Điều khiển được tốc độ của động cơ DC thông qua TB6612 bằng PWM + sử dụng các cổng trùng với sơ đồ minh họa , (tôi điều khiển động cơ với tốc độ từ 0% -> 100% sau đó 100%->0%)

_motor trái

Màu dây / Tên cổng	Chức năng	Nối vào
○ White (1)	Cực 1 Motor	BO1 (TB6612)
● Blue (2)	Encoder VCC	3.3V (STM32)
● Green (3)	Encoder Channel B	B10 STM32 (TIMx_CH2 hoặc GPIO)
● Yellow (4)	Encoder Channel A	B11 STM32 (TIMx_CH1 hoặc GPIO)
● Black (5)	Encoder GND	GND
● Red (6)	Cực 2 Motor	BO2 (TB6612)

#NOTE: khi đấu mạch, phải đảm bảo

_cực âm của Pin, GND của STM32, GND của TB6612 phải được nối đến GND chung, nếu không nó sẽ không chạy.



_Chân STBY (standby) ở TB6612: Công tắc bật/tắt driver -> HIGH -> Driver hoạt động -> động cơ chạy

_Chân BO1 và BO2: dùng để đấu vào 2 cực của động cơ để điều khiển

_motor phải

Màu dây / Tên cổng	Chức năng	Nối vào
○ White (1)	Cực 1 Motor	AO1 (TB6612)
● Blue (2)	Encoder VCC	3.3V (STM32)
● Green (3)	Encoder Channel B	B08 STM32 (TIMx_CH2 hoặc GPIO)
● Yellow (4)	Encoder Channel A	B09 STM32 (TIMx_CH1 hoặc GPIO)
● Black (5)	Encoder GND	GND
● Red (6)	Cực 2 Motor	AO2 (TB6612)

_TB6612:

Cổng TB6612	Nối đến	Cổng TB6612	Nối đến
VM	12V	PWMA	A0
VCC	3.3V	AI1	A2
GND	GND	AI2	A3
A01	WHITE (động cơ R)	STBY	A4 (HIGH/LOW)
A02	RED(động cơ R)	BI1	A5
B02	WHITE (động cơ L)	BI2	A6
B01	RED (động cơ L)	PWMB	A1
GND	GND (STM32)	GND	

Cấu hình CubeMX:

1. RCC

HSE = Crystal/Ceramic Resonator

2. SYS

Debug = Serial Wire

3. Clock Configuration

Mục	Giá trị
HSE	8 MHz
PLL Source	HSE
PLL Mul	×9
SYSCLK	72 MHz
AHB	/1
APB1	/2
APB2	/1
ADC	/6

4. TIM2 (PWM)

Enable:

TIM2

Chọn

Channel	Mode
CH1	PWM Generation CH1
CH2	PWM Generation CH2

Parameter Settings

Thông số	Giá trị
Prescaler	0

Thông số	Giá trị
Counter Period (ARR)	3599
Counter Mode	Up
Clock Division	DIV1
Auto Reload Preload	Enable

Lúc này

72 MHz

↓

ARR = 3599

↓

PWM = 20 kHz

Đây là tần số rất phổ biến cho điều khiển động cơ DC vì nằm ngoài dải nghe của tai người.

5. GPIO

Pin	Mode	Label
PA2	GPIO_Output	AIN1
PA3	GPIO_Output	AIN2
PA4	GPIO_Output	STBY
PA5	GPIO_Output	BIN1
PA6	GPIO_Output	BIN2

Output

Push Pull

Pull

No Pull

Speed

High

6. PWM Pins

CubeMX sẽ tự gán

Pin	Chức năng
PA0	TIM2_CH1
PA1	TIM2_CH2

7. Encoder

Tạm thời

Pin	Mode
PB10	GPIO_Input
PB11	GPIO_Input

8. NVIC

Hiện tại Không cần bật Interrupt

STM32CubeMX SB_PWM_DCMotor.ioc*: STM32F103C8Tx

File Window Help

Home > STM32F103C8Tx > SB_PWM_DCMotor.ioc - Pinout & Configuration > GENERATE CODE

Pinout & Configuration | Clock Configuration | Project Manager | Tools

Software Packs Pinout

Categories A-Z

System Core

- DMA
- ✓ GPIO
- ✓ IWDG
- ✓ NVIC
- ✓ RCC
- ⚠ SYS
- WWDG

Analog >

Timers

- RTC
- TIM1
- ⚠ TIM2
- TIM3
- TIM4

Connectivity >

TIM2 Mode and Configuration

Mode

Slave Mode: Disable

Trigger Source: Disable

Clock Source: Internal Clock

Channel1: PWM Generation CH1

Channel2: Disable

Channel3: Disable

Channel4: Disable

Combined Channels: Disable

☐ Use ETR as Clearing Source

☐ XOR activation

Configuration

Reset Configuration

• NVIC Settings • DMA Settings • GPIO Settings

• Parameter Settings • User Constants

Configure the below parameters :

Search (Ctrl+F)

Counter Settings

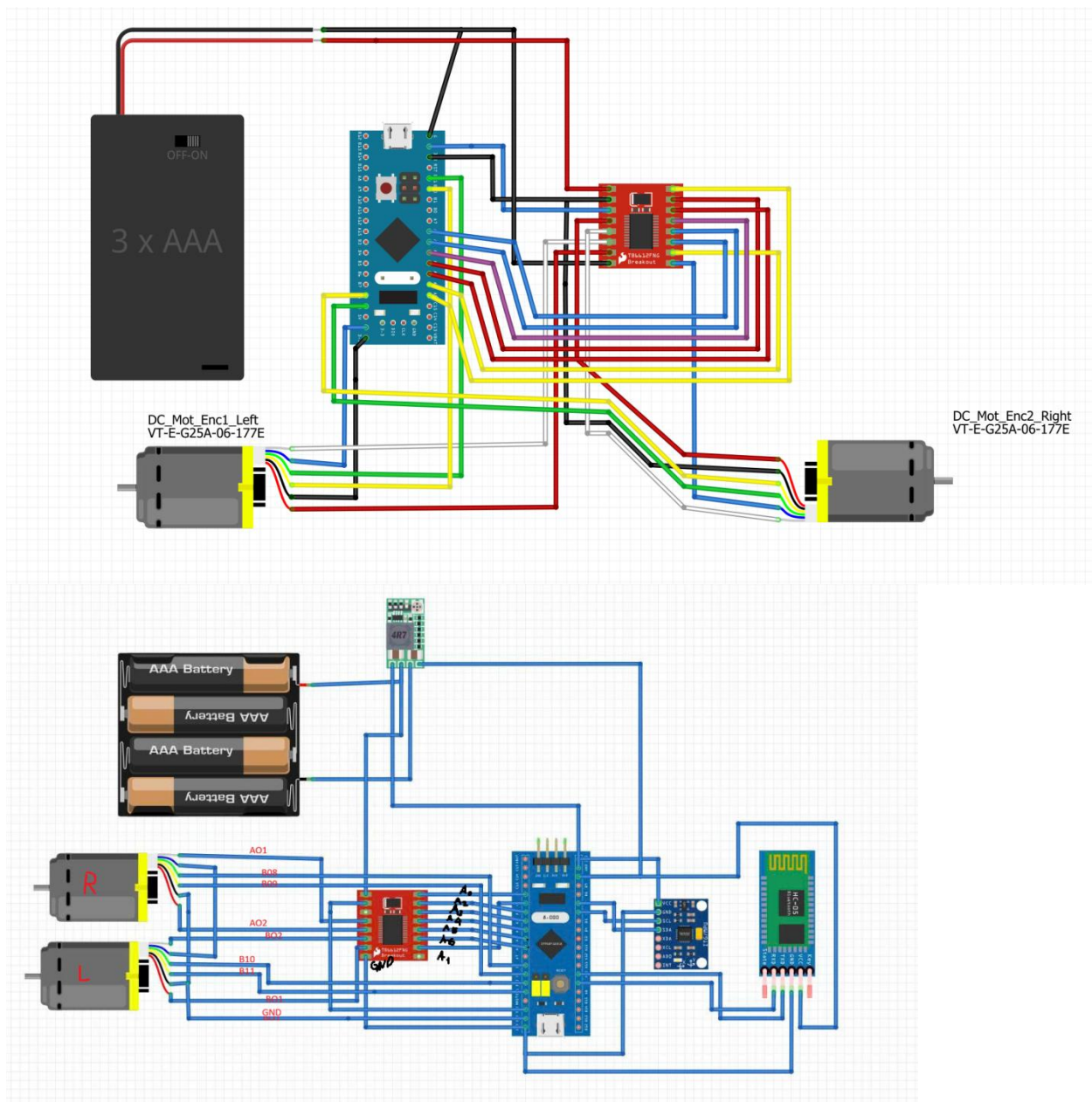
Prescaler (PSC - 16 bits v... 0

Pinout view | System view

Pinout view

Diagram of STM32F103C8Tx LQFP48 package showing pin connections:

- Top: VDD, VSS, PB9, PB8, BOD, PB7, PB6, PB5, PB4, PB3, PB2, PA15, PA14, PA13, PA12, PA11, PA10, PA9, PA8, PA7, PA6, PA5, PA4, PA3, PA2, PA1, PA0, PD0, PD1, NRST, VSSA, VDDA, TIM2_CH1, TIM2_CH2, GPIO_Output, GPIO_Input, GPIO_Output, GPIO_Output, GPIO_Output, GPIO_Output, GPIO_Input, GPIO_Input, VSS, VDD.
- Bottom: VBAT, PC13, PC14, PC15, PD0, PD1, NRST, VSSA, VDDA, TIM2_CH1, TIM2_CH2, GPIO_Output, GPIO_Input, GPIO_Output, GPIO_Output, GPIO_Output, GPIO_Output, GPIO_Input, GPIO_Input, VSS, VDD.
- Left: VDD, VSS, PB9, PB8, BOD, PB7, PB6, PB5, PB4, PB3, PB2, PA15, PA14, PA13, PA12, PA11, PA10, PA9, PA8, PA7, PA6, PA5, PA4, PA3, PA2, PA1, PA0, PD0, PD1, NRST, VSSA, VDDA, TIM2_CH1, TIM2_CH2, GPIO_Output, GPIO_Input, GPIO_Output, GPIO_Output, GPIO_Output, GPIO_Output, GPIO_Input, GPIO_Input, VSS, VDD.
- Right: VDD, VSS, PB9, PB8, BOD, PB7, PB6, PB5, PB4, PB3, PB2, PA15, PA14, PA13, PA12, PA11, PA10, PA9, PA8, PA7, PA6, PA5, PA4, PA3, PA2, PA1, PA0, PD0, PD1, NRST, VSSA, VDDA, TIM2_CH1, TIM2_CH2, GPIO_Output, GPIO_Input, GPIO_Output, GPIO_Output, GPIO_Output, GPIO_Output, GPIO_Input, GPIO_Input, VSS, VDD.



#note: cái hình này chưa hẳn đã đúng, nên kiểm tra lại tính chính xác, bởi vì gần đây đã có sự thay đổi về mặt phần cứng

1. STM32 ↔ TB6612

STM32	Chế độ trong CubeMX	TB6612	Chức năng
PA0	TIM2_CH1 (PWM)	PWMA	Điều khiển tốc độ động cơ phải
PA1	TIM2_CH2 (PWM)	PWMB	Điều khiển tốc độ động cơ trái
PA2	GPIO Output	AIN1	Chiều quay động cơ phải
PA3	GPIO Output	AIN2	Chiều quay động cơ phải
PA4	GPIO Output	STBY	Enable TB6612
PA5	GPIO Output	BIN1	Chiều quay động cơ trái
PA6	GPIO Output	BIN2	Chiều quay động cơ trái
3.3V	Power	VCC	Nguồn logic TB6612
GND	GND	GND	Mass chung

2. TB6612 ↔ Động cơ

Động cơ phải

TB6612	Động cơ
AO1	White
AO2	Red

Động cơ trái

TB6612	Động cơ
BO1	Red
BO2	White

3. Encoder động cơ trái

Dây encoder	STM32	Chức năng
Black	3.3V	VCC
Blue	GND	GND
Yellow	PB11	Channel A
Green	PB10	Channel B

4. Nguồn

Nguồn	TB6612
+12V (hoặc nguồn động cơ của bạn)	VM
3.3V STM32	VCC
GND nguồn	GND
GND STM32	GND

Lưu ý: GND của STM32, TB6612 và nguồn động cơ phải nối chung.

5. Các chân đang được chương trình sử dụng

STM32	Chức năng
PA0	PWM Motor Right
PA1	PWM Motor Left
PA2	AIN1
PA3	AIN2

STM32	Chức năng
PA4	STBY
PA5	BIN1
PA6	BIN2
PB10	Encoder B (chưa sử dụng trong code)
PB11	Encoder A (chưa sử dụng trong code)

6. Trạng thái các chân trong chương trình

Khi quay thuận

Chân	Mức
STBY	HIGH
AIN1	HIGH
AIN2	LOW
BIN1	HIGH
BIN2	LOW
PWMA	PWM 0 → 100%
PWMB	PWM 0 → 100%

Khi quay ngược

Chân	Mức
STBY	HIGH

Chân	Mức
AIN1	LOW
AIN2	HIGH
BIN1	LOW
BIN2	HIGH
PWMA	PWM 0 → 100%
PWMB	PWM 0 → 100%

#MiniTask 2: Việc của bạn là sử dụng được Encoder của động cơ, hiển thị các thông số vị trí của bánh 1 và bánh 2, yêu cầu đầu nối giống với những gì đã thiết lập

<đã xong> (file

#MiniTask3: Việc của bạn là thử điều khiển PID nó, thử tác động một lực quay nhất định rồi yêu cầu nó quay trở về vị trí cũ.

<đã xong>

/* USER CODE BEGIN Header */

/**

*** @file : main.c**

*** @brief : Main program body**

*** @attention**

*** Copyright (c) 2026 STMicroelectronics.**

*** All rights reserved.**

*** This software is licensed under terms that can be found in the LICENSE file**

*** in the root directory of this software component.**

*** If no LICENSE file comes with this software, it is provided AS-IS.**

***/**

/* USER CODE END Header */

/* Includes -----*/

#include "main.h"

/* Private includes -----*/

/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----*/

/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----*/

/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----*/

/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/

TIM_HandleTypeDef htim2;

TIM_HandleTypeDef htim3;

/* USER CODE BEGIN PV */

int pwm = 0;

volatile int32_t encoder_position = 0;

volatile int16_t cnt = 0;

int16_t encoder_last = 0;

int32_t target_position = 0;

/* PID */

float Kp = 0.1f;

float Ki = 0.01f;

float Kd = 0.5f;

float integral = 0.0f;

float previous_error = 0.0f;

float last_error = 0;

float derivative = 0.0f;

float error = 0;

/* USER CODE END PV */

/* Private function prototypes -----*/

void SystemClock_Config(void);

static void MX_GPIO_Init(void);

static void MX_TIM2_Init(void);

static void MX_TIM3_Init(void);

/* USER CODE BEGIN PFP */

void HAL_TIM_MspPostInit(TIM_HandleTypeDef *htim);

void Motor_SetPWM(int pwm);


```
void Encoder_Update(void);
```

```
/* USER CODE END PFP */
```

```
/* Private user code -----*/
```

```
/* USER CODE BEGIN 0 */
```

```
/* USER CODE END 0 */
```

```
/**
```

```
 * @brief The application entry point.
```

```
 * @retval int
```

```
 */
```

```
int main(void)
```

```
{
```

```
/* USER CODE BEGIN 1 */
```

```
/* USER CODE END 1 */
```

```
/* MCU Configuration-----*/
```

```
/* Reset of all peripherals, Initializes the Flash interface and the Systick. */
```

```
HAL_Init();
```

/* USER CODE BEGIN Init */

/* USER CODE END Init */

/* Configure the system clock */

SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */

MX_GPIO_Init();

MX_TIM2_Init();

MX_TIM3_Init();

/* USER CODE BEGIN 2 */

// Khởi động các kênh xung PWM trên TIMER 2

HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);

HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);

// Bật chân STBY để kích hoạt IC driver TB6612 (PA4)

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);

```
HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);  
encoder_last = (int16_t)__HAL_TIM_GET_COUNTER(&htim3);  
Encoder_Update();  
target_position = encoder_position;  
Encoder_Update();
```

```
/* USER CODE END 2 */
```

```
/* Infinite loop */
```

```
/* USER CODE BEGIN WHILE */
```

```
volatile int32_t temp = 0;
```

```
while (1)
```

```
{
```

```
/* USER CODE END WHILE */
```

```
/* USER CODE BEGIN 3 */
```

```
Encoder_Update();
```

```
cnt = (int16_t)__HAL_TIM_GET_COUNTER(&htim3);
```

```
error = target_position - encoder_position;
```

// Khâu Tích phân (I)

integral += error;

// integral Windup

if(integral > 5000) integral = 5000;

if(integral < -5000) integral = -5000;

// Khâu Vi phân (D)

derivative = error - last_error;

// Tính toán đầu ra PID

pwm = (int)(Kp * error + Ki * integral + Kd * derivative);

// CẬP NHẬT SAI SỐ CŨ CHO CHU KỲ SAU (BẮT BUỘC)

last_error = error;

// Vùng chết (Deadband)

if(error > -3 && error < 3)

{

pwm = 0;

}

// Giới hạn điện áp PWM đầu ra chống quá tải hệ thống

if(pwm > 1200) pwm = 1200;

```
if(pwm < -1200) pwm = -1200;
```

```
Motor_SetPWM(pwm);
```

```
HAL_Delay(5);
```

```
}
```

```
/* USER CODE END 3 */
```

```
}
```

```
/**
```

```
 * @brief System Clock Configuration
```

```
 * @retval None
```

```
 */
```

```
void SystemClock_Config(void)
```

```
{
```

```
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
```

```
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
```

```
/** Initializes the RCC Oscillators according to the specified parameters
```

```
 * in the RCC_OscInitTypeDef structure.
```

```
 */
```

```
RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
```

```
RCC_OscInitStruct.HSEState = RCC_HSE_ON;
```

```
RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
```

```
RCC_OscInitStruct.HSIState = RCC_HSI_ON;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
{
    Error_Handler();
}
```

```
/** Initializes the CPU, AHB and APB buses clocks
*/
```

```
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                               |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
```

```
if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
{
    Error_Handler();
}
}
```

```

/**
 * @brief TIM2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM2_Init(void)
{

    /* USER CODE BEGIN TIM2_Init 0 */

    /* USER CODE END TIM2_Init 0 */

    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};
    TIM_OC_InitTypeDef sConfigOC = {0};

    /* USER CODE BEGIN TIM2_Init 1 */

    /* USER CODE END TIM2_Init 1 */
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 0;
    htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim2.Init.Period = 3599;
    htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;

```

```
htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;

if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
{
    Error_Handler();
}

sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;

if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
{
    Error_Handler();
}

if (HAL_TIM_PWM_Init(&htim2) != HAL_OK)
{
    Error_Handler();
}

sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;

sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;

if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) !=
HAL_OK)
{
    Error_Handler();
}

sConfigOC.OCMode = TIM_OCMode_PWM1;

sConfigOC.Pulse = 0;

sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;

sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
```



```

    if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1) !=
    HAL_OK)
    {
        Error_Handler();
    }

    if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_2) !=
    HAL_OK)
    {
        Error_Handler();
    }

    /* USER CODE BEGIN TIM2_Init 2 */

    /* USER CODE END TIM2_Init 2 */

    HAL_TIM_MspPostInit(&htim2);

}

/**
 * @brief TIM3 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM3_Init(void)
{

```

/* USER CODE BEGIN TIM3_Init 0 */

/* USER CODE END TIM3_Init 0 */

TIM_Encoder_InitTypeDef sConfig = {0};

TIM_MasterConfigTypeDef sMasterConfig = {0};

/* USER CODE BEGIN TIM3_Init 1 */

/* USER CODE END TIM3_Init 1 */

htim3.Instance = TIM3;

htim3.Init.Prescaler = 0;

htim3.Init.CounterMode = TIM_COUNTERMODE_UP;

htim3.Init.Period = 65535;

htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;

htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;

sConfig.EncoderMode = TIM_ENCODERMODE_TI12;

sConfig.IC1Polarity = TIM_ICPOLARITY_RISING;

sConfig.IC1Selection = TIM_ICSELECTION_DIRECTTI;

sConfig.IC1Prescaler = TIM_ICPSC_DIV1;

sConfig.IC1Filter = 10;

sConfig.IC2Polarity = TIM_ICPOLARITY_RISING;

sConfig.IC2Selection = TIM_ICSELECTION_DIRECTTI;

sConfig.IC2Prescaler = TIM_ICPSC_DIV1;

```

sConfig.IC2Filter = 10;

if (HAL_TIM_Encoder_Init(&htim3, &sConfig) != HAL_OK)
{
    Error_Handler();
}

sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) !=
HAL_OK)
{
    Error_Handler();
}

/* USER CODE BEGIN TIM3_Init 2 */


/* USER CODE END TIM3_Init 2 */

}


/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */

static void MX_GPIO_Init(void)
{

```

```

GPIO_InitTypeDef GPIO_InitStructure = {0};

/* USER CODE BEGIN MX_GPIO_Init_1 */

/* USER CODE END MX_GPIO_Init_1 */

/* GPIO Ports Clock Enable */

__HAL_RCC_GPIOD_CLK_ENABLE();
__HAL_RCC_GPIOA_CLK_ENABLE();
__HAL_RCC_GPIOB_CLK_ENABLE();

/*Configure GPIO pin Output Level */
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2|GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5
                  |GPIO_PIN_6, GPIO_PIN_RESET);

/*Configure GPIO pins : PA2 PA3 PA4 PA5
                        PA6 */
GPIO_InitStructure.Pin = GPIO_PIN_2|GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5
                        |GPIO_PIN_6;
GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStructure.Pull = GPIO_NOPULL;
GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

/* USER CODE BEGIN MX_GPIO_Init_2 */

```

```

/* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

// Hàm x? l? ng?t ngo?i d? d?m xung Encoder t? ch?n PB10 v? PB11

void Motor_SetPWM(int pwm)
{
    // Giới hạn giá trị PWM đầu ra theo cấu hình Timer (Period = 3599)
    if(pwm > 3599) pwm = 3599;
    if(pwm < -3599) pwm = -3599;

    if(pwm <= 0) // Quay thuận khi pwm dương hoặc bằng 0
    {
        // QUAY THUAN: IN1 = 1, IN2 = 0
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);

        // Nạp giá trị PWM dương trực tiếp
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, pwm);
    }
    else // Quay nghịch khi pwm < 0

```

```

{
    // QUAY NGHỊCH: IN1 = 0, IN2 = 1
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);

    // ĐỔI DẤU SỐ ÂM THÀNH SỐ DƯƠNG (Thêm dấu trừ)
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, -pwm);
}
}

void Encoder_Update(void)
{
    int16_t now;
    int16_t delta;

    now = (int16_t)__HAL_TIM_GET_COUNTER(&htim3);

    delta = now - encoder_last;

    encoder_last = now;

    encoder_position += delta;
}

/* USER CODE END 4 */

```

```

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 *        where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)

```

```

{
    /* USER CODE BEGIN 6 */

    /* User can add his own implementation to report the file name and line number,
       ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */

    /* USER CODE END 6 */
}

#endif /* USE_FULL_ASSERT */

```

#NOTE: đổi chân encoder motor trái thành PB4 PB5

#MiniTask4: Tìm ra bài toán động học và làm sao cho nó đứng được.

#MiniTask5: Tuning bằng module blue tooth, học cách sử dụng và Tuning

Kết luận

Mình khá tự tin (trên 99%) rằng động cơ của bạn là:

- Encoder: 11 PPR
- Gear ratio: 30:1
- CPR ở trục ra: 1320 counts/rev

Sai số 9 count trên tổng 26400 count chỉ khoảng 0.03%, hoàn toàn có thể do:

- Quay tay hơi quá hoặc thiếu một chút.
- Rung cơ khí.
- Sai số khi dùng đúng vạch đánh dấu.

#MINI TASK 3.2: encoder speed: sử dụng encoder để tính tốc độ RPM để phục vụ cho bài toán động học giữa MPU và Động cơ.

Module 1: Encoder Speed

Mục tiêu

Mỗi 10 ms tính được:

Position (count)

↓

Delta Count

↓

RPM

↓

Filtered RPM

Bước 1. Cấu hình CubeMX

TIM3

Giữ nguyên như hiện tại

Mode:

Encoder Mode

Prescaler = 0

Period = 65535

Encoder Mode = TI12

IC1 Filter = 10

IC2 Filter = 10

TIM4

Tạo timer định kỳ.

Ví dụ Clock:

72 MHz

Ta muốn

10 ms

=> 100 Hz

Chọn

PSC = 7199

ARR = 99

Vì

72 MHz

↓

$72000000/(7199+1)$

=

10000 Hz

↓

$10000/(99+1)$

=

100 Hz

=

10 ms

Sau đó

Enable Update Interrupt

Bước 2. Tạo biến

Ngay đầu main.c

typedef struct

{

int32_t position;

int32_t last_position;

int32_t delta;

```
float rpm;
```

```
float rpm_filter;
```

```
}Encoder_t;
```

```
Encoder_t encoder;
```

Bước 3.

Thêm

```
#define ENCODER_CPR    1320.0f
```

```
#define SAMPLE_TIME    0.01f
```

```
#define RPM_FACTOR (60.0f/(ENCODER_CPR*SAMPLE_TIME))
```

Compiler sẽ tính sẵn

RPM_FACTOR

=

4.54545

Bước 4.

Khởi động

```
HAL_TIM_Encoder_Start(&htim3,TIM_CHANNEL_ALL);
```

```
HAL_TIM_Base_Start_IT(&htim4);
```

Bước 5.

Viết hàm

```
void Encoder_Update(void)
```

```
{  
    encoder.position = (int16_t)_HAL_TIM_GET_COUNTER(&htim3);  
  
    encoder.delta =  
    encoder.position -  
    encoder.last_position;  
  
    encoder.last_position =  
    encoder.position;  
  
    encoder.rpm =  
    encoder.delta * RPM_FACTOR;  
  
    encoder.rpm_filter =  
    0.8f*encoder.rpm_filter +  
    0.2f*encoder.rpm;  
}
```

#Side Quest: Hiện tại việc Debug PID rất phiền, nên chúng ta chuyển sang sử dụng UART để debug -> học sử dụng HC-05 bluetooth trước

HC-05		
STATE		
RXD	PA9	
TXD	PA10	
GND	GND	
VCC	5V	
EN		

PA9 PA10

Bước 1: Kết nối phần cứng

Nếu dùng USART1 của Blue Pill:

HC-05 STM32F103

VCC 5V

GND GND

TXD PA10 (USART1_RX)

RXD PA9 (USART1_TX)

Lưu ý

TX luôn nối RX

RX luôn nối TX

STM32 PA9 -----> RX HC05

STM32 PA10 <----- TX HC05

Bước 2: CubeMX

Mở CubeMX.

USART1

Chọn

Mode

Asynchronous

Baudrate

9600

Word Length

8 Bits

Parity

None

Stop Bits

1

Hardware Flow Control

None

Mode

TX and RX

DMA

Disable

Interrupt

Disable

Hiện tại chúng ta chỉ truyền dữ liệu nên chưa cần ngắt.

Bước 3: Clock

Để USART ổn định:

SYSCLK = 72 MHz

Nếu trước đó Motor và Encoder hoạt động bình thường thì giữ nguyên.

Bước 4: Kiểm tra HC-05

Cấp nguồn.

Quan sát LED.

Trường hợp 1

LED nhấp nháy

• • • • •

=> Chưa kết nối Bluetooth.

Đây là trạng thái bình thường.

Trường hợp 2

LED nhấp chậm

• • •

=> Đã kết nối điện thoại.

Bước 5: Ghép nối điện thoại

Bật Bluetooth.

Tìm

HC-05

Nếu hỏi mật khẩu

1234

hoặc

0000

Theo datasheet, mã PIN mặc định có thể là 0000 trong chế độ hoạt động và 1234 sau khi khôi phục mặc định bằng AT+ORGL, tùy firmware của module.

Bước 6

Cài ứng dụng

Serial Bluetooth Terminal

(Đây là ứng dụng mình thường dùng để debug UART qua HC-05.)

Bước 7

Sau khi Pair thành công

Kết nối tới HC-05.

Nếu LED chuyển sang nhấp nháy chậm

=> Thành công.

Bước 8

Chưa cần viết code.

Mình muốn kiểm tra phần cứng trước.

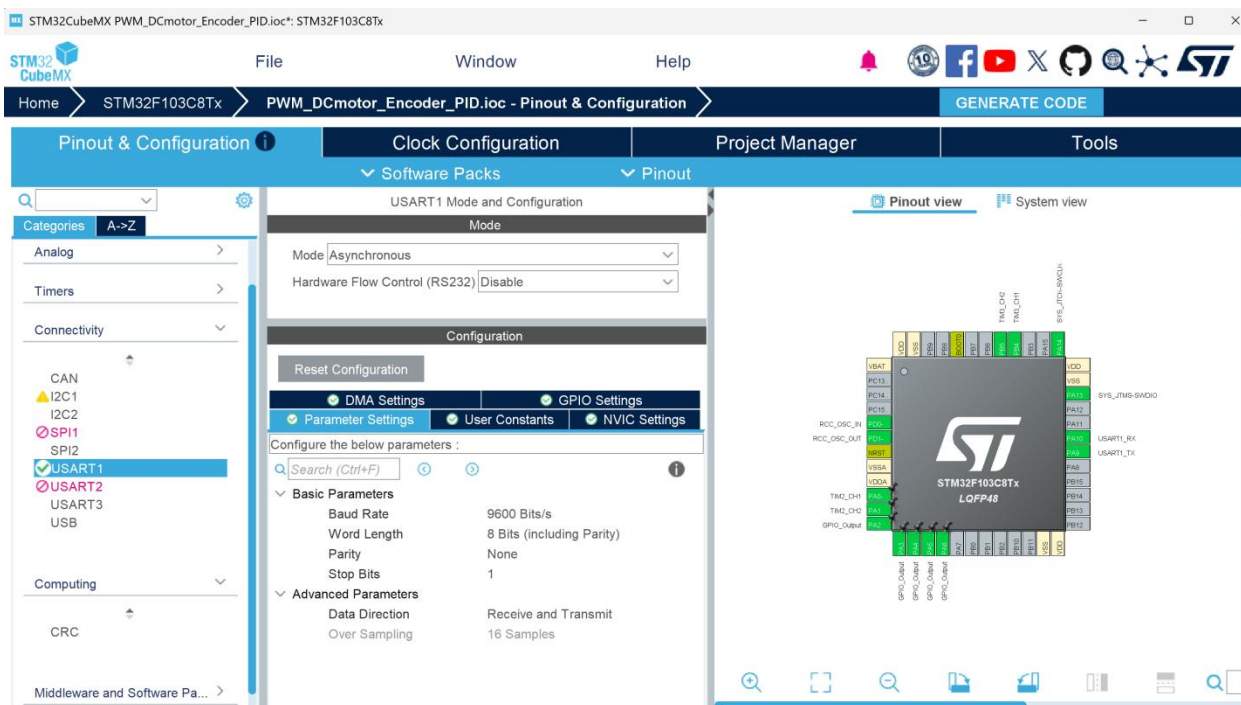
Bạn hãy gửi cho mình:

1. Ảnh module HC-05 của bạn (hai mặt nếu được) để xem đây là bản ZS-040, FC-114 hay loại khác.
2. File .ioc hiện tại của dự án STM32 (hoặc ảnh cấu hình CubeMX phần USART1).

Sau khi mình kiểm tra cấu hình, chúng ta sẽ sang Bài 2, chỉ với khoảng 10 dòng code để STM32 gửi:

Hello STM32

Nếu điện thoại nhận được dòng này, tức là toàn bộ đường truyền STM32 → UART → HC-05 → Bluetooth đã hoạt động chính xác. Sau đó chúng ta mới bắt đầu xây dựng thư viện Bluetooth.



Kiểm tra phần cứng

Hãy nối đúng như sau:

HC-05 STM32

HC-05 STM32

VCC 5V

GND GND

TXD PA10 (RX1)

RXD PA9 (TX1)

⚠ Lưu ý quan trọng:

- TX của HC-05 → RX của STM32
- RX của HC-05 ← TX của STM32

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
```

```
{  
    if(huart->Instance == USART1)  
    {  
        HAL_UART_Transmit(&huart1,&rxChar,1,100);  
        if(rxChar == '\r' || rxChar == '\n')  
        {  
            rxBuffer[rxIndex] = '\0';  
  
            rxIndex = 0;  
        }  
    }  
}
```

```

        cmdReady = 1;
    }
    else
    {
        if(rxIndex < sizeof(rxBuffer)-1)
        {
            rxBuffer[rxIndex++] = rxChar;
        }
    }

    HAL_UART_Receive_IT(&huart1, &rxChar, 1);
}
}

```

#MINI Task viết giao thức để giao tiếp với STM32 bằng UART Bluetooth

Mục tiêu là khi nhập lệnh thì STM32 phải phản hồi lại, từ đó phục vụ cho việc tuning không dây

_làm cho nó phản hồi được

🔍 Phân tích nguyên nhân (Root Cause Analysis)

1. Xung đột ngắt & HAL Deadlock giữa TIM4 và UART (Nguyên nhân chính)

- Hiện trạng: TIM4 ngắt mỗi 10ms (100Hz). Trong callback `HAL_TIM_PeriodElapsedCallback`, bạn gọi `Control_Loop()`, và trong đó có dòng:

C

`Motor_SetPWM(3599);` // Hoặc các tính toán liên tục

- Lỗi trong Callback Echo:

C

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
```

```
{
```

```
    if(huart->Instance == USART1)
```

```
    {
```

```
        // DỪNG HAL_MAX_DELAY TRONG INTERRUPT CONTEXT!
```

```
        HAL_UART_Transmit(&huart1, (uint8_t*) &rxChar, 1, HAL_MAX_DELAY);
```

```
        HAL_UART_Receive_IT(&huart1, (uint8_t*) &rxChar, 1);
```

```
    }
```

```
}
```

- Hậu quả:
 - `HAL_UART_Transmit` với `HAL_MAX_DELAY` là hàm blocking (chờ lặp). Gọi hàm blocking ngay bên trong ISR (Interrupt Service Routine) của UART là điều cấm kỵ. Nếu đường truyền bị nhiễu hoặc HC-05 chưa sẵn sàng nhận, MCU sẽ bị khóa cứng (Deadlock) ngay trong ISR.
 - Mức ưu tiên ngắt (Priority): Nếu TIM4 ngắt đè lên UART khi đang truyền dữ liệu, UART State của HAL sẽ bị đơ (`HAL_BUSY`), khiến các lệnh `HAL_UART_Receive_IT` tiếp theo bị từ chối trả về `HAL_ERROR`.

2. Thiếu ngắt NVIC cho USART1

- Khi cấu hình bằng STM32CubeMX, nếu chỉ chọn USART1 Asynchronous mà chưa tích chọn "USART1 global interrupt" trong mục NVIC Settings, lệnh HAL_UART_Receive_IT() sẽ chỉ chạy đúng 1 lần rồi im lặng mãi mãi vì phần cứng không bao giờ nhảy vào ISR.

🔧 Từng bước khắc phục & Kiểm chứng (Debug Steps)

Bước 1: Kiểm tra cấu hình NVIC trong CubeMX

Mở file .ioc trong STM32CubeIDE:

1. Vào Connectivity → USART1.
2. Sang tab NVIC Settings.
3. Tích chọn USART1 global interrupt → Enabled.
4. Generate Code lại (Ctrl + S).

Bước 2: Sửa lại code Echo Interrupt an toàn (Không Blocking)

Sửa lại hàm HAL_UART_RxCpltCallback để loại bỏ HAL_MAX_DELAY. Thay vào đó, dùng hằng số Timeout ngắn (ví dụ 10ms), hoặc gửi bằng ngắt/xử lý ngoài main:

C

```
/* USER CODE BEGIN 4 */
```

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
```

```
{
```

```
    if(huart->Instance == USART1)
```

```
    {
```

```
        // 1. Phản hồi ngay ký tự vừa nhận với Timeout an toàn (10ms) thay vì  
        HAL_MAX_DELAY
```

```
        HAL_UART_Transmit(&huart1, (uint8_t*)&rxChar, 1, 10);
```

```

// 2. Kích hoạt lại ngắt nhận 1 byte tiếp theo ngay lập tức
HAL_UART_Receive_IT(&huart1, (uint8_t*)&rxChar, 1);
}
}

```

/* USER CODE END 4 */

Bước 3: Xử lý Tràn bộ đệm & Tự khôi phục lỗi UART (Overrun Error Handling)

Một lỗi rất phổ biến khiến UART ngưng nhận dữ liệu trên STM32 HAL là Overrun Error (ORE). Khi ký tự đến quá nhanh mà MCU chưa kịp đọc, HAL sẽ khóa ngắt UART.

Thêm hàm callback xử lý lỗi này vào cuối file main.c để tự khôi phục ngắt nếu có sự cố nhiễu/tràn phím:

C

```

void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART1)
    {
        // Xóa cờ lỗi ORE/NE/FE nếu có
        __HAL_UART_CLEAR_OREFLAG(huart);
        __HAL_UART_CLEAR_NEFLAG(huart);
        __HAL_UART_CLEAR_FEFLAG(huart);

        // Kích hoạt lại ngắt nhận
        HAL_UART_Receive_IT(&huart1, (uint8_t*)&rxChar, 1);
    }
}

```

}

Kịch bản Test Echo trên điện thoại

1. Nạp firmware đã cập nhật ở trên vào Blue Pill.
2. Mở ứng dụng Serial Bluetooth trên điện thoại (ví dụ: *Serial Bluetooth Terminal* trên Android).
3. Kết nối với HC-05.
4. Gõ từng ký tự (ví dụ: H, E, L, L, O).
5. Kỳ vọng: Mỗi khi bạn bấm 1 phím trên điện thoại, ký tự đó sẽ lập tức xuất hiện ngược lại trên màn hình app.

Nếu Echo hoạt động thành công, ta đã chứng minh 100%: HC-05 ✓ →

USART1 Hardware ✓ → NVIC Interrupt ✓ → HAL Callback ✓

Bạn hãy thực hiện kiểm tra Bước 1 & Bước 2 trên phần cứng. Cho mình biết kết quả Echo (có nhận được ký tự phản hồi từng chữ cái hay chưa) để chúng ta bước sang Tầng Parser & Xây dựng CLI Ring Buffer nhé!

Các lệnh mình đề xuất

Đây là những lệnh đủ dùng cho cả dự án robot.

Lệnh	Ý nghĩa
HELP	Liệt kê lệnh
PWM 1500	Chạy PWM trực tiếp
RPM 60	Đặt tốc độ
STOP	Dừng
ENC	In encoder
POS	In vị trí
KP 30	Đổi Kp
KI 0.2	Đổi Ki
KD 0.5	Đổi Kd
PID	In thông số PID

Mình đề xuất cấu trúc như sau

```
Core
├── Inc
│   ├── motor.h
│   ├── encoder.h
│   ├── pid.h
│   ├── cli.h
│   ├── mpu6050.h
│   └── balance.h
└── Src
    ├── motor.c
    ├── encoder.c
    ├── pid.c
    ├── cli.c
    ├── mpu6050.c
    └── balance.c
```

```
/* USER CODE BEGIN Header */
```

```
/**
```

```
*****
```

```
* @file      : main.c
```

```
* @brief     : Main program body
```

```
*****
```

```
* @attention
```

```
*
```

```
* Copyright (c) 2026 STMicroelectronics.
```

```
* All rights reserved.
```

```
*
```

```
* This software is licensed under terms that can be found in the LICENSE file
```

```
* in the root directory of this software component.
```

```
* If no LICENSE file comes with this software, it is provided AS-IS.
```

```
*
```

```
*****
```

```
*/
```

```
/* USER CODE END Header */
```

```
/* Includes -----*/
```

```
#include "main.h"
```

```

/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */
typedef struct
{
    float Kp;
    float Ki;
    float Kd;

    float Ts;

    float setpoint;
    float feedback;
    float output;

    float error;
    float last_error;

    float integral;
    float derivative;

    // Các thành phần kết quả nhân P, I, D riêng biệt
    float P_term;
    float I_term;
    float D_term;

    float outMax;
    float outMin;

    float iMax;
    float iMin;
} PID_t;

typedef enum {

```

```

    MODE_MANUAL_PWM = 0,
    MODE_PID_SPEED = 1
} ControlMode_t;

typedef enum {
    PRINT_OFF = 0,
    PRINT_MONITOR, // In dạng chữ chi tiết có cả Kp, Ki, Kd
    PRINT_PLOT     // In dạng CSV để vẽ đồ thị
} PrintMode_t;
/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */
#define ENCODER_CPR 1320.0f
#define SAMPLE_TIME 0.01f
#define RPM_FACTOR (60.0f / (ENCODER_CPR * SAMPLE_TIME))
/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/
TIM_HandleTypeDef htim2;
TIM_HandleTypeDef htim3;
TIM_HandleTypeDef htim4;

UART_HandleTypeDef huart1;

/* USER CODE BEGIN PV */
// Control & Mode Flags
volatile ControlMode_t control_mode = MODE_MANUAL_PWM;
volatile PrintMode_t print_mode = PRINT_OFF;
volatile uint8_t print_flag = 0; // Cờ báo đến giờ in (100ms)

int manual_pwm = 0;
volatile int32_t encoder_position = 0;
volatile int16_t encoder_now = 0;
volatile int16_t encoder_old = 0;

```

```
volatile int16_t encoder_delta = 0;  
volatile float encoder_rpm = 0.0f;
```

```
float target_rpm = 0.0f;  
volatile uint32_t timer_cnt = 0;
```

```
// UART CLI Variables  
volatile uint8_t rxChar;  
char rxBuffer[32];  
uint8_t rxIndex = 0;  
volatile uint8_t cmdReady = 0;
```

```
PID_t Speed_PID;  
PID_t Angle_PID;  
/* USER CODE END PV */
```

```
/* Private function prototypes -----*/  
void SystemClock_Config(void);  
static void MX_GPIO_Init(void);  
static void MX_TIM2_Init(void);  
static void MX_TIM3_Init(void);  
static void MX_TIM4_Init(void);  
static void MX_USART1_UART_Init(void);  
/* USER CODE BEGIN PFP */  
void HAL_TIM_MspPostInit(TIM_HandleTypeDef *htim);  
void Motor_SetPWM(int pwm);  
void Encoder_Update(void);  
void PID_Init(PID_t *pid);  
float PID_Update(PID_t *pid);  
void Process_Command(void);  
void Process_Print(void);  
/* USER CODE END PFP */
```

```
/* Private user code -----*/  
/* USER CODE BEGIN 0 */
```

```
/* USER CODE END 0 */
```

```
/**  
 * @brief The application entry point.
```

```

* @retval int
*/
int main(void)
{

    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_TIM2_Init();
    MX_TIM3_Init();
    MX_TIM4_Init();
    MX_USART1_UART_Init();
    /* USER CODE BEGIN 2 */
    // Khởi động các kênh xung PWM trên TIMER 2
    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);

    // Bật chân STBY để kích hoạt IC driver TB6612 (PA4)
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);

    // Khởi động Encoder TIM3

```

```

HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);
encoder_old = (int16_t)_HAL_TIM_GET_COUNTER(&htim3);

// Cấu hình PID Speed mặc định
Speed_PID.Kp = 40.0f;
Speed_PID.Ki = 0.0f;
Speed_PID.Kd = 0.0f;
Speed_PID.Ts = SAMPLE_TIME;
Speed_PID.outMax = 3599.0f;
Speed_PID.outMin = -3599.0f;
Speed_PID.iMax = 1000.0f;
Speed_PID.iMin = -1000.0f;
PID_Init(&Speed_PID);

// Bắt đầu Timer ngắt chu kỳ 10ms (100Hz)
HAL_TIM_Base_Start_IT(&htim4);

// Kích hoạt ngắt nhận UART1 (1 byte)
HAL_UART_Receive_IT(&huart1, (uint8_t*)&rxChar, 1);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    // 1. Xử lý lệnh từ UART / Bluetooth
    if (cmdReady)
    {
        cmdReady = 0;
        Process_Command();
    }

    // 2. Xuất dữ liệu Monitor / Plotter ngoài luồng ngắt (chu kỳ 100ms)
    Process_Print();
}
/* USER CODE END 3 */
}

```

```

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Initializes the RCC Oscillators according to the specified parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
     */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                                   |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
    {
        Error_Handler();
    }
}

```



```

/**
 * @brief TIM2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM2_Init(void)
{

    /* USER CODE BEGIN TIM2_Init 0 */

    /* USER CODE END TIM2_Init 0 */

    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};
    TIM_OC_InitTypeDef sConfigOC = {0};

    /* USER CODE BEGIN TIM2_Init 1 */

    /* USER CODE END TIM2_Init 1 */
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 0;
    htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim2.Init.Period = 3599;
    htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
    if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
    {
        Error_Handler();
    }
    sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
    if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
    {
        Error_Handler();
    }
    if (HAL_TIM_PWM_Init(&htim2) != HAL_OK)
    {
        Error_Handler();
    }
    sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
    sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;

```

```

    if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) !=
    HAL_OK)
    {
        Error_Handler();
    }
    sConfigOC.OCMode = TIM_OCMODE_PWM1;
    sConfigOC.Pulse = 0;
    sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
    sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
    if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1) !=
    HAL_OK)
    {
        Error_Handler();
    }
    if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_2) !=
    HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN TIM2_Init 2 */

    /* USER CODE END TIM2_Init 2 */
    HAL_TIM_MspPostInit(&htim2);

}

/**
 * @brief TIM3 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM3_Init(void)
{

    /* USER CODE BEGIN TIM3_Init 0 */

    /* USER CODE END TIM3_Init 0 */

    TIM_Encoder_InitTypeDef sConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};

```

```

/* USER CODE BEGIN TIM3_Init 1 */

/* USER CODE END TIM3_Init 1 */
htim3.Instance = TIM3;
htim3.Init.Prescaler = 0;
htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
htim3.Init.Period = 65535;
htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
sConfig.EncoderMode = TIM_ENCODERMODE_TI12;
sConfig.IC1Polarity = TIM_ICPOLARITY_RISING;
sConfig.IC1Selection = TIM_ICSELECTION_DIRECTTI;
sConfig.IC1Prescaler = TIM_ICPSC_DIV1;
sConfig.IC1Filter = 10;
sConfig.IC2Polarity = TIM_ICPOLARITY_RISING;
sConfig.IC2Selection = TIM_ICSELECTION_DIRECTTI;
sConfig.IC2Prescaler = TIM_ICPSC_DIV1;
sConfig.IC2Filter = 10;
if (HAL_TIM_Encoder_Init(&htim3, &sConfig) != HAL_OK)
{
    Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) !=
HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM3_Init 2 */

/* USER CODE END TIM3_Init 2 */

}

/**
 * @brief TIM4 Initialization Function
 * @param None
 * @retval None

```

```

*/
static void MX_TIM4_Init(void)
{

    /* USER CODE BEGIN TIM4_Init 0 */

    /* USER CODE END TIM4_Init 0 */

    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};

    /* USER CODE BEGIN TIM4_Init 1 */

    /* USER CODE END TIM4_Init 1 */
    htim4.Instance = TIM4;
    htim4.Init.Prescaler = 7199;
    htim4.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim4.Init.Period = 99;
    htim4.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim4.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_Base_Init(&htim4) != HAL_OK)
    {
        Error_Handler();
    }
    sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
    if (HAL_TIM_ConfigClockSource(&htim4, &sClockSourceConfig) != HAL_OK)
    {
        Error_Handler();
    }
    sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
    sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
    if (HAL_TIMEx_MasterConfigSynchronization(&htim4, &sMasterConfig) !=
    HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN TIM4_Init 2 */

    /* USER CODE END TIM4_Init 2 */

```

```

}

/**
 * @brief USART1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_USART1_UART_Init(void)
{

    /* USER CODE BEGIN USART1_Init 0 */

    /* USER CODE END USART1_Init 0 */

    /* USER CODE BEGIN USART1_Init 1 */

    /* USER CODE END USART1_Init 1 */
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;
    huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart1.Init.OverSampling = UART_OVERSAMPLING_16;
    if (HAL_UART_Init(&huart1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN USART1_Init 2 */

    /* USER CODE END USART1_Init 2 */

}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */

```

```

static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */

    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOD_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2|GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5
        |GPIO_PIN_6, GPIO_PIN_RESET);

    /*Configure GPIO pins : PA2 PA3 PA4 PA5
        PA6 */
    GPIO_InitStruct.Pin = GPIO_PIN_2|GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5
        |GPIO_PIN_6;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    /* USER CODE BEGIN MX_GPIO_Init_2 */

    /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

void Motor_SetPWM(int pwm)
{
    if(pwm > 3599) pwm = 3599;
    if(pwm < -3599) pwm = -3599;

    if(pwm >= 0)
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
    }
}

```

```

    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, pwm);
}
else
{
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, -pwm);
}
}

```

```

void Encoder_Update(void)

```

```

{
    encoder_now = (int16_t)__HAL_TIM_GET_COUNTER(&htim3);
    encoder_delta = encoder_now - encoder_old;
    encoder_old = encoder_now;

    encoder_position += encoder_delta;
    encoder_rpm = (float)encoder_delta * RPM_FACTOR;
}

```

```

void Control_Loop(void)

```

```

{
    Encoder_Update();

    if (control_mode == MODE_PID_SPEED)
    {
        Speed_PID.feedback = encoder_rpm;
        Speed_PID.setpoint = target_rpm;
        float out = PID_Update(&Speed_PID);
        Motor_SetPWM((int)out);
    }
    else
    {
        Motor_SetPWM(manual_pwm);
    }
}

```

```

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)

```

```

{

```

```

if(htim->Instance == TIM4)
{
    timer_cnt++;
    Control_Loop(); // Tính toán PID mỗi 10ms

    // Bật cờ xuất dữ liệu UART sau mỗi 100ms (10 x 10ms)
    static uint8_t print_cnt = 0;
    if (++print_cnt >= 10)
    {
        print_cnt = 0;
        if (print_mode != PRINT_OFF)
        {
            print_flag = 1;
        }
    }
}
}

```

```

void PID_Init(PID_t *pid)
{
    pid->error = 0.0f;
    pid->last_error = 0.0f;
    pid->integral = 0.0f;
    pid->derivative = 0.0f;
    pid->P_term = 0.0f;
    pid->I_term = 0.0f;
    pid->D_term = 0.0f;
    pid->output = 0.0f;
    pid->setpoint = 0.0f;
    pid->feedback = 0.0f;
}

```

```

float PID_Update(PID_t *pid)
{
    pid->error = pid->setpoint - pid->feedback;

    // Tích phân (Integral)
    pid->integral += pid->error * pid->Ts;
    if(pid->integral > pid->iMax) pid->integral = pid->iMax;
    if(pid->integral < pid->iMin) pid->integral = pid->iMin;
}

```



```

// Vi phân (Derivative)
pid->derivative = (pid->error - pid->last_error) / pid->Ts;

// Tính riêng từng thành phần
pid->P_term = pid->Kp * pid->error;
pid->I_term = pid->Ki * pid->integral;
pid->D_term = pid->Kd * pid->derivative;

// Tổng ngõ ra
pid->output = pid->P_term + pid->I_term + pid->D_term;

// Bảo hòa ngõ ra PWM
if(pid->output > pid->outMax) pid->output = pid->outMax;
if(pid->output < pid->outMin) pid->output = pid->outMin;

pid->last_error = pid->error;

return pid->output;
}

/*
=====
=====
* UART MONITOR / PLOTTER PRINT PROCESSOR (Chạy trong vòng lặp while(1))
*
=====
===== */
void Process_Print(void)
{
    if (!print_flag) return;
    print_flag = 0; // Reset cờ

    char msg[160];

    if (print_mode == PRINT_MONITOR)
    {
        // Hiển thị cả Kp Ki Kd | Target, Real, Err | P, I, D | Output PWM
        sprintf(msg, "[Kp:%.1f Ki:%.1f Kd:%.1f] | T:%5.1f R:%5.1f E:%5.1f | P:%6.1f I:%6.1f D:%6.1f | OUT:%5.0f\r\n",

```

```

        Speed_PID.Kp, Speed_PID.Ki, Speed_PID.Kd,
        target_rpm, encoder_rpm, Speed_PID.error,
        Speed_PID.P_term, Speed_PID.I_term, Speed_PID.D_term,
        Speed_PID.output);

    HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 30);
}
else if (print_mode == PRINT_PLOT)
{
    // Định dạng CSV dành cho Serial Plotter vẽ 5 đường đồng thời:
    // Cú pháp: Target, RPM, P_term, I_term, D_term
    sprintf(msg, "%.1f,%.1f,%.1f,%.1f,%.1f\r\n",
        target_rpm,
        encoder_rpm,
        Speed_PID.P_term,
        Speed_PID.I_term,
        Speed_PID.D_term);
    HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 20);
}
}

/*
=====
=====
* CLI COMMAND PROCESSOR
*
=====
===== */
void Process_Command(void)
{
    char msg[128];

    // --- Lệnh HELP ---
    if(strcmp(rxBuffer, "HELP") == 0)
    {
        char *help_text =
            "\r\n===== COMMAND LIST =====\r\n"
            "HELP      : Show command menu\r\n"
            "PWM xxx   : Set manual PWM (-3599 to 3599)\r\n"
            "RPM xxx   : Set target RPM (Enable PID)\r\n"

```

```

"STOP      : Stop motor & reset mode\r\n"
"STATUS    : Show Mode, RPM & Encoder\r\n"
"MONITOR   : Stream Kp,Ki,Kd | T,R,E | P,I,D | OUT\r\n"
"PLOT      : Stream CSV (Target,RPM,P,I,D) for Plotter\r\n"
"STOPMON   : Stop streaming data\r\n"
"ENC       : Show Encoder position\r\n"
"KP xxx    : Set Speed PID Kp\r\n"
"KI xxx    : Set Speed PID Ki\r\n"
"KD xxx    : Set Speed PID Kd\r\n"
"===== \r\n";

```

```

    HAL_UART_Transmit(&huart1, (uint8_t*)help_text, strlen(help_text), 100);
}
// --- Lệnh PWM xxx ---
else if(strncmp(rxBuffer, "PWM ", 4) == 0)
{
    int val = 0;
    if(sscanf(rxBuffer + 4, "%d", &val) == 1)
    {
        control_mode = MODE_MANUAL_PWM;
        manual_pwm = val;
        Motor_SetPWM(manual_pwm);

        sprintf(msg, "\r\n[MODE] Manual PWM = %d\r\n", manual_pwm);
        HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
    }
}
// --- Lệnh RPM xxx ---
else if(strncmp(rxBuffer, "RPM ", 4) == 0)
{
    float val = 0.0f;
    if(sscanf(rxBuffer + 4, "%f", &val) == 1)
    {
        control_mode = MODE_PID_SPEED;
        target_rpm = val;

        sprintf(msg, "\r\n[MODE] PID Speed Target = %.2f RPM\r\n", target_rpm);
        HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
    }
}
}

```

```

// --- Lệnh MONITOR ---
else if(strcmp(rxBuffer, "MONITOR") == 0)
{
    print_mode = PRINT_MONITOR;
    char *m = "\r\n[MONITOR ON - Full PID Inspection]\r\n";
    HAL_UART_Transmit(&huart1, (uint8_t*)m, strlen(m), 50);
}
// --- Lệnh PLOT ---
else if(strcmp(rxBuffer, "PLOT") == 0)
{
    print_mode = PRINT_PLOT;
}
// --- Lệnh STOPMON ---
else if(strcmp(rxBuffer, "STOPMON") == 0)
{
    print_mode = PRINT_OFF;
    char *m = "\r\n[MONITOR OFF]\r\n";
    HAL_UART_Transmit(&huart1, (uint8_t*)m, strlen(m), 50);
}
// --- Lệnh STOP ---
else if(strcmp(rxBuffer, "STOP") == 0)
{
    control_mode = MODE_MANUAL_PWM;
    manual_pwm = 0;
    target_rpm = 0.0f;
    Motor_SetPWM(0);

    sprintf(msg, "\r\nMotor Stopped!\r\n");
    HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
}
// --- Lệnh STATUS ---
else if(strcmp(rxBuffer, "STATUS") == 0)
{
    sprintf(msg, "\r\nMode: %s | PWM: %d | Target RPM: %.1f | Current
RPM: %.1f | Pos: %ld\r\n",
        (control_mode == MODE_PID_SPEED) ? "PID_SPEED" : "MANUAL_PWM",
        manual_pwm, target_rpm, encoder_rpm, (long)encoder_position);
    HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
}
// --- Lệnh ENC ---

```

```

else if(strcmp(rxBuffer, "ENC") == 0 || strcmp(rxBuffer, "POS") == 0)
{
    sprintf(msg, "\r\nEncoder Pos = %ld\r\n", (long)encoder_position);
    HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
}
// --- Lệnh chỉnh PID Kp ---
else if(strncmp(rxBuffer, "KP ", 3) == 0)
{
    float val = 0.0f;
    if(sscanf(rxBuffer + 3, "%f", &val) == 1)
    {
        Speed_PID.Kp = val;
        sprintf(msg, "\r\nKp set to %.3f\r\n", Speed_PID.Kp);
        HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
    }
}
// --- Lệnh chỉnh PID Ki ---
else if(strncmp(rxBuffer, "KI ", 3) == 0)
{
    float val = 0.0f;
    if(sscanf(rxBuffer + 3, "%f", &val) == 1)
    {
        Speed_PID.Ki = val;
        sprintf(msg, "\r\nKi set to %.3f\r\n", Speed_PID.Ki);
        HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
    }
}
// --- Lệnh chỉnh PID Kd ---
else if(strncmp(rxBuffer, "KD ", 3) == 0)
{
    float val = 0.0f;
    if(sscanf(rxBuffer + 3, "%f", &val) == 1)
    {
        Speed_PID.Kd = val;
        sprintf(msg, "\r\nKd set to %.3f\r\n", Speed_PID.Kd);
        HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
    }
}
// --- Lệnh không xác định ---
else

```

```

    {
        sprintf(msg, "\r\nUnknown Command: %s\r\n", rxBuffer);
        HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
    }
}

/*
=====
=====
* UART CLI RECEIVER & CALLBACKS
*
=====
===== */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART1)
    {
        // 1. Echo ký tự vừa gõ
        HAL_UART_Transmit(&huart1, (uint8_t*)&rxChar, 1, 10);

        // 2. Kiểm tra ký tự kết thúc dòng
        if (rxChar == '\r' || rxChar == '\n')
        {
            if (rxIndex > 0)
            {
                rxBuffer[rxIndex] = '\0';
                cmdReady = 1;
                rxIndex = 0;
            }
        }
        else
        {
            if (rxIndex < (sizeof(rxBuffer) - 1))
            {
                rxBuffer[rxIndex++] = rxChar;
            }
            else
            {
                rxIndex = 0; // Chống tràn bộ đệm
            }
        }
    }
}

```

```

    }

    // 3. Tiếp tục lắng nghe ký tự tiếp theo
    HAL_UART_Receive_IT(&huart1, (uint8_t*)&rxChar, 1);
}
}

void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART1)
    {
        __HAL_UART_CLEAR_OREFLAG(huart);
        __HAL_UART_CLEAR_NEFLAG(huart);
        __HAL_UART_CLEAR_FEFLAG(huart);

        HAL_UART_Receive_IT(&huart1, (uint8_t*)&rxChar, 1);
    }
}

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 *        where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number

```

```
* @retval None
*/
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```


#TASK 4: Angle PID and MPU control + filter

#dùng được MPU:

MPU	STM32	
VCC	5V	
GND	GND	
SCL	PB6 (I2C1_SCL)	
SDA	PB7 (I2C1_SDA)	

Cấu hình CubeMX

Hãy thiết lập:

I2C1

- Mode: I2C
- SCL: PB6
- SDA: PB7
- Clock Speed: 100000 Hz
- Addressing Mode: 7-bit
- Dual Address: Disable
- General Call: Disable
- No Stretch: Disable

RCC

- HSE nếu bạn đang dùng thạch anh ngoài (giữ nguyên như project hiện tại).

NVIC

- Chưa cần bật Interrupt cho I2C.

DMA

- Chưa cần.

Sau đó chúng ta sẽ làm đúng quy trình

Bước 1: Kiểm tra thiết bị có tồn tại trên bus I2C không.

HAL_I2C_IsDeviceReady(...)

↓

Nếu thành công:

MPU6050 Found

↓

Bước 2: Đọc thanh ghi WHO_AM_I.

↓

Bước 3: Đánh thức MPU6050.

↓

Bước 4: Đọc gia tốc và con quay.

↓

Bước 5: Tính góc.

Bước 1:

B1: Thêm biến

Trong `/* USER CODE BEGIN PV */`
`I2C_HandleTypeDef hi2c1;`

`uint8_t txBuf[100];`

`HAL_StatusTypeDef mpuStatus;`

Lưu ý: `hi2c1` thường đã được CubeMX tạo trong `main.c`. Nếu đã có thì không khai báo lại, chỉ thêm `txBuf` và `mpuStatus`.

B2: Địa chỉ I2C của MPU6050

Thêm vào phần `/* USER CODE BEGIN PD */`

`#define MPU6050_ADDR (0x68 << 1)`

Tại sao lại `<< 1`?

- Địa chỉ thật của MPU6050 là 0x68
- HAL của STM32 yêu cầu địa chỉ 7-bit dịch trái 1 bit

nên:

`0x68 << 1 = 0xD0`

B3: Sau `MX_I2C1_Init()`;

Trong `main()`

`MX_GPIO_Init();`

`MX_USART1_UART_Init();`

`MX_I2C1_Init();`

thêm:

`HAL_Delay(100);`

`mpuStatus = HAL_I2C_IsDeviceReady(&hi2c1, MPU6050_ADDR, 3, 100);`

`if(mpuStatus == HAL_OK)`

`{`
`sprintf((char*)txBuf, "MPU6050 Found!\r\n");`

`}`

`else`

`{`
`sprintf((char*)txBuf, "MPU6050 Not Found!\r\n");`
`}`

```
HAL_UART_Transmit(&huart1, txBuf, strlen((char*)txBuf), HAL_MAX_DELAY);
```

Bước 2: Đọc thanh ghi WHO_AM_I

Thanh ghi này giống như "chứng minh thư" của MPU6050.

- Địa chỉ thanh ghi: 0x75
- Giá trị mong đợi: 0x68

Thêm biến

Trong USER CODE BEGIN PV

```
uint8_t who_am_i;
```

Đọc thanh ghi

Ngay sau khi in "MPU Found" (vẫn trong USER CODE BEGIN 2):

```
HAL_I2C_Mem_Read(&hi2c1,  
                 MPU6050_ADDR,  
                 0x75,  
                 I2C_MEMADD_SIZE_8BIT,  
                 &who_am_i,  
                 1,  
                 HAL_MAX_DELAY);
```

```
sprintf((char*)txBuf, "WHO_AM_I = 0x%02X\r\n", who_am_i);
```

```
HAL_UART_Transmit(&huart1,  
                 txBuf,  
                 strlen((char*)txBuf),  
                 HAL_MAX_DELAY);
```

Kết quả mong đợi

Bluetooth sẽ hiện:

MPU6050 Found!

WHO_AM_I = 0x68

Nếu đúng 0x68 thì gần như chắc chắn chúng ta đang giao tiếp với đúng MPU6050.

Đánh thức (Wake Up) MPU6050

Bước 1: Tạo biến

Trong USER CODE BEGIN PV:

```
uint8_t data;
```

Bước 2: Ghi vào thanh ghi

Ngay sau khi đọc WHO_AM_I thành công:

```
data = 0x00;
```

```
HAL_I2C_Mem_Write(&hi2c1,  
    MPU6050_ADDR,  
    0x6B,  
    I2C_MEMADD_SIZE_8BIT,  
    &data,  
    1,  
    HAL_MAX_DELAY);
```

Đoạn này nghĩa là:

Ghi giá trị 0x00 vào thanh ghi 0x6B.

Bước 3: Kiểm tra lại

Để chắc chắn việc ghi thành công, ta đọc lại chính thanh ghi đó:

```
HAL_I2C_Mem_Read(&hi2c1,  
    MPU6050_ADDR,  
    0x6B,  
    I2C_MEMADD_SIZE_8BIT,  
    &data,  
    1,  
    HAL_MAX_DELAY);
```

```
sprintf((char*)txBuf, "PWR_MGMT_1 = 0x%02X\r\n", data);
```

```
HAL_UART_Transmit(&huart1,  
    txBuf,  
    strlen((char*)txBuf),  
    HAL_MAX_DELAY);
```

Nếu mọi thứ đúng, Bluetooth sẽ hiện:

MPU6050 Found!

WHO_AM_I = 0x68

PWR_MGMT_1 = 0x00

MPU6050 lưu dữ liệu ở đâu?

Các thanh ghi từ 0x3B đến 0x48 chứa toàn bộ dữ liệu cảm biến:

Thanh ghi Dữ liệu

0x3B-0x3C Accel X

0x3D-0x3E Accel Y

0x3F-0x40 Accel Z

Thanh ghi Dữ liệu

0x41-0x42 Temperature

0x43-0x44 Gyro X

0x45-0x46 Gyro Y

0x47-0x48 Gyro Z

Mỗi giá trị đều là 16-bit, nên cần đọc 2 byte rồi ghép lại.

#code đang gặp lỗi nè:

```
/* USER CODE BEGIN Header */  
/**  
*****  
* @file      : main.c  
* @brief     : Main program body  
*****  
* @attention  
*  
* Copyright (c) 2026 STMicroelectronics.  
* All rights reserved.  
*  
* This software is licensed under terms that can be found in the LICENSE file  
* in the root directory of this software component.  
* If no LICENSE file comes with this software, it is provided AS-IS.  
*  
*****  
*/  
/* USER CODE END Header */  
/* Includes -----*/  
#include "main.h"  
  
/* Private includes -----*/  
/* USER CODE BEGIN Includes */  
#include <string.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <math.h>  
/* USER CODE END Includes */  
  
/* Private typedef -----*/
```

```

/* USER CODE BEGIN PTD */
typedef struct
{
    float Kp;
    float Ki;
    float Kd;

    float Ts;

    float setpoint;
    float feedback;
    float output;

    float error;
    float last_error;

    float integral;
    float derivative;

    // Các thành phần kết quả nhân P, I, D riêng biệt
    float P_term;
    float I_term;
    float D_term;

    float outMax;
    float outMin;

    float iMax;
    float iMin;
} PID_t;

typedef enum {
    MODE_MANUAL_PWM = 0,
    MODE_PID_SPEED = 1
} ControlMode_t;

typedef enum {
    PRINT_OFF = 0,
    PRINT_MONITOR, // In dạng chữ chi tiết có cả Kp, Ki, Kd
    PRINT_PLOT     // In dạng CSV để vẽ đồ thị

```

```

} PrintMode_t;
/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */
#define ENCODER_CPR 1320.0f
#define SAMPLE_TIME 0.01f
#define RPM_FACTOR (60.0f / (ENCODER_CPR * SAMPLE_TIME))
#define MPU6050_ADDR (0x68 << 1)
#define M_PI 3.14159265358979323846
#define DT 0.01f
/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/
I2C_HandleTypeDef hi2c1;

TIM_HandleTypeDef htim2;
TIM_HandleTypeDef htim3;
TIM_HandleTypeDef htim4;

UART_HandleTypeDef huart1;

/* USER CODE BEGIN PV */
// Control & Mode Flags
volatile ControlMode_t control_mode = MODE_MANUAL_PWM;
volatile PrintMode_t print_mode = PRINT_OFF;
volatile uint8_t print_flag = 0; // Cờ báo đến giờ in (100ms)

int manual_pwm = 0;
volatile int32_t encoder_position = 0;
volatile int16_t encoder_now = 0;
volatile int16_t encoder_old = 0;
volatile int16_t encoder_delta = 0;
volatile float encoder_rpm = 0.0f;

```



```
float target_rpm = 0.0f;  
volatile uint32_t timer_cnt = 0;
```

```
// UART CLI Variables  
volatile uint8_t rxChar;  
char rxBuffer[32];  
uint8_t rxIndex = 0;  
volatile uint8_t cmdReady = 0;
```

```
PID_t Speed_PID;  
PID_t Angle_PID;
```

```
// MPU Variables  
uint8_t txBuf[100];  
HAL_StatusTypeDef mpuStatus;  
uint8_t who_am_i;  
uint8_t data;
```

```
uint8_t mpuData[14];
```

```
int16_t AccX;  
int16_t AccY;  
int16_t AccZ;
```

```
int16_t GyroX;  
int16_t GyroY;  
int16_t GyroZ;
```

```
int16_t Temp;
```

```
float Ax;  
float Ay;  
float Az;  
float Gx;  
float Gy;  
float Gz;  
float AccAngle = 0.0f;
```

```
float PitchAcc = 0.0f;  
float Roll = 0.0f;
```

```

float RollAcc = 0.0f;
float RollGyro = 0.0f;
float GxOffset = 0.0f;
/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_TIM2_Init(void);
static void MX_TIM3_Init(void);
static void MX_TIM4_Init(void);
static void MX_USART1_UART_Init(void);
static void MX_I2C1_Init(void);
/* USER CODE BEGIN PFP */
void HAL_TIM_MspPostInit(TIM_HandleTypeDef *htim);
void Motor_SetPWM(int pwm);
void Encoder_Update(void);
void PID_Init(PID_t *pid);
float PID_Update(PID_t *pid);
void Process_Command(void);
void Process_Print(void);
void MPU6050_GyroCalibration(void);
/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{

/* USER CODE BEGIN 1 */

/* USER CODE END 1 */

```

```

/* MCU Configuration-----*/

/* Reset of all peripherals, Initializes the Flash interface and the Systick. */
HAL_Init();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_TIM2_Init();
MX_TIM3_Init();
MX_TIM4_Init();
MX_USART1_UART_Init();
MX_I2C1_Init();
/* USER CODE BEGIN 2 */
// Khởi động các kênh xung PWM trên TIMER 2
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);

// Bật chân STBY để kích hoạt IC driver TB6612 (PA4)
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);

// Khởi động Encoder TIM3
HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);
encoder_old = (int16_t)_HAL_TIM_GET_COUNTER(&htim3);

// Cấu hình PID Speed mặc định
Speed_PID.Kp = 40.0f;
Speed_PID.Ki = 0.0f;
Speed_PID.Kd = 0.0f;
Speed_PID.Ts = SAMPLE_TIME;

```

```

Speed_PID.outMax = 3599.0f;
Speed_PID.outMin = -3599.0f;
Speed_PID.iMax = 1000.0f;
Speed_PID.iMin = -1000.0f;
PID_Init(&Speed_PID);

// Bắt đầu Timer ngắt chu kỳ 10ms (100Hz)
HAL_TIM_Base_Start_IT(&htim4);

// Kích hoạt ngắt nhận UART1 (1 byte)
HAL_UART_Receive_IT(&huart1, (uint8_t*)&rxChar, 1);

data = 0x00;

        HAL_I2C_Mem_Write(&hi2c1,MPU6050_ADDR,
        0x6B,
        I2C_MEMADD_SIZE_8BIT,
        &data,
        1,
        HAL_MAX_DELAY);

        MPU6050_GyroCalibration();

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    // 1. Xử lý lệnh từ UART / Bluetooth
    if (cmdReady)
    {
        cmdReady = 0;
        Process_Command();
    }

    // 2. Xuất dữ liệu Monitor / Plotter ngoài luồng ngắt (chu kỳ 100ms)

```

Process_Print();

HAL_Delay(100);

**mpuStatus = HAL_I2C_IsDeviceReady(&hi2c1, MPU6050_ADDR, 3,
100);
/***

if(mpuStatus == HAL_OK)

{

sprintf((char*)txBuf, "MPU6050 Found!\r\n");

}

else

{

sprintf((char*)txBuf, "MPU6050 Not Found!\r\n");

}

**HAL_UART_Transmit(&huart1, txBuf, strlen((char*)txBuf),
HAL_MAX_DELAY);
*/**

**HAL_I2C_Mem_Read(&hi2c1,MPU6050_ADDR,0x75,I2C_MEMADD_SIZE_8BIT,
&who_am_i,1,HAL_MAX_DELAY);
/***

sprintf((char*)txBuf, "WHO_AM_I = 0x%02X\r\n", who_am_i);

**HAL_UART_Transmit(&huart1,txBuf,strlen((char*)txBuf),HAL_MAX_DELAY);
*/**

**HAL_I2C_Mem_Read(&hi2c1,
MPU6050_ADDR,
0x6B,
I2C_MEMADD_SIZE_8BIT,**

```

        &data,
        1,
        HAL_MAX_DELAY);
/*
sprintf((char*)txBuf, "PWR_MGMT_1 = 0x%02X\r\n", data);
HAL_UART_Transmit(&huart1,
        txBuf,
        strlen((char*)txBuf),
        HAL_MAX_DELAY);
*/

```

```

        HAL_I2C_Mem_Read(&hi2c1,
        MPU6050_ADDR,
        0x3B,
        I2C_MEMADD_SIZE_8BIT,
        mpuData,
        14,
        HAL_MAX_DELAY);

```

```

    AccX = (int16_t)(mpuData[0] << 8 | mpuData[1]);
    AccY = (int16_t)(mpuData[2] << 8 | mpuData[3]);
    AccZ = (int16_t)(mpuData[4] << 8 | mpuData[5]);

```

```

    Temp = (int16_t)(mpuData[6] << 8 | mpuData[7]);

```

```

    GyroX = (int16_t)(mpuData[8] << 8 | mpuData[9]);
    GyroY = (int16_t)(mpuData[10] << 8 | mpuData[11]);
    GyroZ = (int16_t)(mpuData[12] << 8 | mpuData[13]);

```

```

    Ax = (float)AccX / 16384.0f;
    Ay = (float)AccY / 16384.0f;
    Az = (float)AccZ / 16384.0f;
    Gx = (float)GyroX / 131.0f - GxOffset;
    Gy = (float)GyroY / 131.0f;
    Gz = (float)GyroZ / 131.0f;

```

```

    PitchAcc = atan2f(Ax,sqrtf(Ay*Ay + Az*Az))*180.0f/M_PI;
    RollAcc = atan2f(Ay, Az) * 180.0f / M_PI;
    RollGyro = Roll + Gx * DT;

```

```

        /* Complementary Filter */
        Roll = 0.98f * RollGyro + 0.02f * RollAcc;

        /*sprintf((char*)txBuf,
            "PitchAcc = %.2f | RollAcc = %.2f | Roll = %.2f\r\n",
            PitchAcc, RollAcc, Roll);*/

        sprintf((char*)txBuf,
            "RollAcc=%.2f Gx=%.2f Roll=%.2f\r\n",
            RollAcc,
            Gx,
            Roll);

        HAL_UART_Transmit(&huart1,
            txBuf,
            strlen((char *)txBuf),
            HAL_MAX_DELAY);

    }

    /* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Initializes the RCC Oscillators according to the specified parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;

```

```

RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
{
    Error_Handler();
}

/** Initializes the CPU, AHB and APB buses clocks
*/
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                               |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
{
    Error_Handler();
}
}

/**
 * @brief I2C1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_I2C1_Init(void)
{
    /* USER CODE BEGIN I2C1_Init 0 */

    /* USER CODE END I2C1_Init 0 */

    /* USER CODE BEGIN I2C1_Init 1 */

    /* USER CODE END I2C1_Init 1 */
    hi2c1.Instance = I2C1;
    hi2c1.Init.ClockSpeed = 100000;

```



```

hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
hi2c1.Init.OwnAddress1 = 0;
hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
hi2c1.Init.OwnAddress2 = 0;
hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
if (HAL_I2C_Init(&hi2c1) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN I2C1_Init 2 */

/* USER CODE END I2C1_Init 2 */

}

/**
 * @brief TIM2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM2_Init(void)
{
    /* USER CODE BEGIN TIM2_Init 0 */

    /* USER CODE END TIM2_Init 0 */

    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};
    TIM_OC_InitTypeDef sConfigOC = {0};

    /* USER CODE BEGIN TIM2_Init 1 */

    /* USER CODE END TIM2_Init 1 */
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 0;
    htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim2.Init.Period = 3599;

```

```

htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
{
    Error_Handler();
}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
{
    Error_Handler();
}
if (HAL_TIM_PWM_Init(&htim2) != HAL_OK)
{
    Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) !=
HAL_OK)
{
    Error_Handler();
}
sConfigOC.OCMode = TIM_OCMODE_PWM1;
sConfigOC.Pulse = 0;
sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1) !=
HAL_OK)
{
    Error_Handler();
}
if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_2) !=
HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM2_Init 2 */

/* USER CODE END TIM2_Init 2 */
HAL_TIM_MspPostInit(&htim2);

```

```
}
```

```
/**
```

```
 * @brief TIM3 Initialization Function
```

```
 * @param None
```

```
 * @retval None
```

```
 */
```

```
static void MX_TIM3_Init(void)
```

```
{
```

```
 /* USER CODE BEGIN TIM3_Init 0 */
```

```
 /* USER CODE END TIM3_Init 0 */
```

```
 TIM_Encoder_InitTypeDef sConfig = {0};
```

```
 TIM_MasterConfigTypeDef sMasterConfig = {0};
```

```
 /* USER CODE BEGIN TIM3_Init 1 */
```

```
 /* USER CODE END TIM3_Init 1 */
```

```
 htim3.Instance = TIM3;
```

```
 htim3.Init.Prescaler = 0;
```

```
 htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
```

```
 htim3.Init.Period = 65535;
```

```
 htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
```

```
 htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
```

```
 sConfig.EncoderMode = TIM_ENCODERMODE_TI12;
```

```
 sConfig.IC1Polarity = TIM_ICPOLARITY_RISING;
```

```
 sConfig.IC1Selection = TIM_ICSELECTION_DIRECTTI;
```

```
 sConfig.IC1Prescaler = TIM_ICPSC_DIV1;
```

```
 sConfig.IC1Filter = 10;
```

```
 sConfig.IC2Polarity = TIM_ICPOLARITY_RISING;
```

```
 sConfig.IC2Selection = TIM_ICSELECTION_DIRECTTI;
```

```
 sConfig.IC2Prescaler = TIM_ICPSC_DIV1;
```

```
 sConfig.IC2Filter = 10;
```

```
 if (HAL_TIM_Encoder_Init(&htim3, &sConfig) != HAL_OK)
```

```
 {
```

```
     Error_Handler();
```

```
 }
```

```

sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) !=
HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM3_Init 2 */

/* USER CODE END TIM3_Init 2 */

}

/**
 * @brief TIM4 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM4_Init(void)
{

    /* USER CODE BEGIN TIM4_Init 0 */

    /* USER CODE END TIM4_Init 0 */

    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};

    /* USER CODE BEGIN TIM4_Init 1 */

    /* USER CODE END TIM4_Init 1 */
    htim4.Instance = TIM4;
    htim4.Init.Prescaler = 7199;
    htim4.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim4.Init.Period = 99;
    htim4.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim4.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_Base_Init(&htim4) != HAL_OK)
    {
        Error_Handler();
    }

```

```

}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim4, &sClockSourceConfig) != HAL_OK)
{
    Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim4, &sMasterConfig) !=
HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM4_Init 2 */

/* USER CODE END TIM4_Init 2 */

}

/**
 * @brief USART1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_USART1_UART_Init(void)
{
    /* USER CODE BEGIN USART1_Init 0 */

    /* USER CODE END USART1_Init 0 */

    /* USER CODE BEGIN USART1_Init 1 */

    /* USER CODE END USART1_Init 1 */
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;

```

```

huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
huart1.Init.OverSampling = UART_OVERSAMPLING_16;
if (HAL_UART_Init(&huart1) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN USART1_Init 2 */

/* USER CODE END USART1_Init 2 */

}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */

    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOD_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2|GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5
        |GPIO_PIN_6, GPIO_PIN_RESET);

    /*Configure GPIO pins : PA2 PA3 PA4 PA5
        PA6 */
    GPIO_InitStruct.Pin = GPIO_PIN_2|GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5
        |GPIO_PIN_6;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;

```

```

HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

/* USER CODE BEGIN MX_GPIO_Init_2 */

/* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

void Motor_SetPWM(int pwm)
{
    if(pwm > 3599) pwm = 3599;
    if(pwm < -3599) pwm = -3599;

    if(pwm >= 0)
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, pwm);
    }
    else
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, -pwm);
    }
}

void Encoder_Update(void)
{
    encoder_now = (int16_t)__HAL_TIM_GET_COUNTER(&htim3);
    encoder_delta = encoder_now - encoder_old;
    encoder_old = encoder_now;

    encoder_position += encoder_delta;
    encoder_rpm = (float)encoder_delta * RPM_FACTOR;
}

void Control_Loop(void)
{

```

Encoder_Update();

```
if (control_mode == MODE_PID_SPEED)  
{  
    Speed_PID.feedback = encoder_rpm;  
    Speed_PID.setpoint = target_rpm;  
    float out = PID_Update(&Speed_PID);  
    Motor_SetPWM((int)out);  
}  
else  
{  
    Motor_SetPWM(manual_pwm);  
}  
}
```

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)  
{  
    if(htim->Instance == TIM4)  
    {  
        timer_cnt++;  
        Control_Loop(); // Tính toán PID mỗi 10ms  
  
        // Bật cờ xuất dữ liệu UART sau mỗi 100ms (10 x 10ms)  
        static uint8_t print_cnt = 0;  
        if (++print_cnt >= 10)  
        {  
            print_cnt = 0;  
            if (print_mode != PRINT_OFF)  
            {  
                print_flag = 1;  
            }  
        }  
    }  
}
```

```
void PID_Init(PID_t *pid)  
{  
    pid->error = 0.0f;  
    pid->last_error = 0.0f;  
    pid->integral = 0.0f;  
}
```



```

    pid->derivative = 0.0f;
    pid->P_term = 0.0f;
    pid->I_term = 0.0f;
    pid->D_term = 0.0f;
    pid->output = 0.0f;
    pid->setpoint = 0.0f;
    pid->feedback = 0.0f;
}

float PID_Update(PID_t *pid)
{
    pid->error = pid->setpoint - pid->feedback;

    // Tích phân (Integral)
    pid->integral += pid->error * pid->Ts;
    if(pid->integral > pid->iMax) pid->integral = pid->iMax;
    if(pid->integral < pid->iMin) pid->integral = pid->iMin;

    // Vi phân (Derivative)
    pid->derivative = (pid->error - pid->last_error) / pid->Ts;

    // Tính riêng từng thành phần
    pid->P_term = pid->Kp * pid->error;
    pid->I_term = pid->Ki * pid->integral;
    pid->D_term = pid->Kd * pid->derivative;

    // Tổng ngõ ra
    pid->output = pid->P_term + pid->I_term + pid->D_term;

    // Bảo hòa ngõ ra PWM
    if(pid->output > pid->outMax) pid->output = pid->outMax;
    if(pid->output < pid->outMin) pid->output = pid->outMin;

    pid->last_error = pid->error;

    return pid->output;
}

```

```

/*
=====
=====
* UART MONITOR / PLOTTER PRINT PROCESSOR (Chạy trong vòng lặp while(1))
*
=====
===== */
void Process_Print(void)
{
    if (!print_flag) return;
    print_flag = 0; // Reset cờ

    char msg[160];

    if (print_mode == PRINT_MONITOR)
    {
        // Hiển thị cả Kp Ki Kd | Target, Real, Err | P, I, D | Output PWM
        sprintf(msg, "[Kp:%.1f Ki:%.1f Kd:%.1f] | T:%5.1f R:%5.1f E:%5.1f | P:%6.1f
I:%6.1f D:%6.1f | OUT:%5.0f\r\n",
            Speed_PID.Kp, Speed_PID.Ki, Speed_PID.Kd,
            target_rpm, encoder_rpm, Speed_PID.error,
            Speed_PID.P_term, Speed_PID.I_term, Speed_PID.D_term,
            Speed_PID.output);

        HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 30);
    }
    else if (print_mode == PRINT_PLOT)
    {
        // Định dạng CSV dành cho Serial Plotter vẽ 5 đường đồng thời:
        // Cú pháp: Target, RPM, P_term, I_term, D_term
        sprintf(msg, "%.1f,%.1f,%.1f,%.1f,%.1f\r\n",
            target_rpm,
            encoder_rpm,
            Speed_PID.P_term,
            Speed_PID.I_term,
            Speed_PID.D_term);
        HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 20);
    }
}
}

```

```

/*
=====
=====
* CLI COMMAND PROCESSOR
*
=====
===== */
void Process_Command(void)
{
    char msg[128];

    // --- Lệnh HELP ---
    if(strcmp(rxBuffer, "HELP") == 0)
    {
        char *help_text =
            "\r\n===== COMMAND LIST =====\r\n"
            "HELP      : Show command menu\r\n"
            "PWM xxx    : Set manual PWM (-3599 to 3599)\r\n"
            "RPM xxx    : Set target RPM (Enable PID)\r\n"
            "STOP      : Stop motor & reset mode\r\n"
            "STATUS     : Show Mode, RPM & Encoder\r\n"
            "MONITOR    : Stream Kp,Ki,Kd | T,R,E | P,I,D | OUT\r\n"
            "PLOT       : Stream CSV (Target,RPM,P,I,D) for Plotter\r\n"
            "STOPMON    : Stop streaming data\r\n"
            "ENC        : Show Encoder position\r\n"
            "KP xxx     : Set Speed PID Kp\r\n"
            "KI xxx     : Set Speed PID Ki\r\n"
            "KD xxx     : Set Speed PID Kd\r\n"
            "===== \r\n";

        HAL_UART_Transmit(&huart1, (uint8_t*)help_text, strlen(help_text), 100);
    }

    // --- Lệnh PWM xxx ---
    else if(strncmp(rxBuffer, "PWM ", 4) == 0)
    {
        int val = 0;
        if(sscanf(rxBuffer + 4, "%d", &val) == 1)
        {
            control_mode = MODE_MANUAL_PWM;
            manual_pwm = val;
        }
    }
}

```

```

    Motor_SetPWM(manual_pwm);

    sprintf(msg, "\r\n[MODE] Manual PWM = %d\r\n", manual_pwm);
    HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
}
}
// --- Lệnh RPM xxx ---
else if(strncmp(rxBuffer, "RPM ", 4) == 0)
{
    float val = 0.0f;
    if(sscanf(rxBuffer + 4, "%f", &val) == 1)
    {
        control_mode = MODE_PID_SPEED;
        target_rpm = val;

        sprintf(msg, "\r\n[MODE] PID Speed Target = %.2f RPM\r\n", target_rpm);
        HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
    }
}
// --- Lệnh MONITOR ---
else if(strcmp(rxBuffer, "MONITOR") == 0)
{
    print_mode = PRINT_MONITOR;
    char *m = "\r\n[MONITOR ON - Full PID Inspection]\r\n";
    HAL_UART_Transmit(&huart1, (uint8_t*)m, strlen(m), 50);
}
// --- Lệnh PLOT ---
else if(strcmp(rxBuffer, "PLOT") == 0)
{
    print_mode = PRINT_PLOT;
}
// --- Lệnh STOPMON ---
else if(strcmp(rxBuffer, "STOPMON") == 0)
{
    print_mode = PRINT_OFF;
    char *m = "\r\n[MONITOR OFF]\r\n";
    HAL_UART_Transmit(&huart1, (uint8_t*)m, strlen(m), 50);
}
// --- Lệnh STOP ---
else if(strcmp(rxBuffer, "STOP") == 0)

```

```

{
    control_mode = MODE_MANUAL_PWM;
    manual_pwm = 0;
    target_rpm = 0.0f;
    Motor_SetPWM(0);

    sprintf(msg, "\r\nMotor Stopped!\r\n");
    HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
}
// --- Lệnh STATUS ---
else if(strcmp(rxBuffer, "STATUS") == 0)
{
    sprintf(msg, "\r\nMode: %s | PWM: %d | Target RPM: %.1f | Current
RPM: %.1f | Pos: %ld\r\n",
        (control_mode == MODE_PID_SPEED) ? "PID_SPEED" : "MANUAL_PWM",
        manual_pwm, target_rpm, encoder_rpm, (long)encoder_position);
    HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
}
// --- Lệnh ENC ---
else if(strcmp(rxBuffer, "ENC") == 0 || strcmp(rxBuffer, "POS") == 0)
{
    sprintf(msg, "\r\nEncoder Pos = %ld\r\n", (long)encoder_position);
    HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
}
// --- Lệnh chỉnh PID Kp ---
else if(strncmp(rxBuffer, "KP ", 3) == 0)
{
    float val = 0.0f;
    if(sscanf(rxBuffer + 3, "%f", &val) == 1)
    {
        Speed_PID.Kp = val;
        sprintf(msg, "\r\nKp set to %.3f\r\n", Speed_PID.Kp);
        HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
    }
}
// --- Lệnh chỉnh PID Ki ---
else if(strncmp(rxBuffer, "KI ", 3) == 0)
{
    float val = 0.0f;
    if(sscanf(rxBuffer + 3, "%f", &val) == 1)

```

```

    {
        Speed_PID.Ki = val;
        sprintf(msg, "\r\nKi set to %.3f\r\n", Speed_PID.Ki);
        HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
    }
}
// --- Lệnh chỉnh PID Kd ---
else if(strncmp(rxBuffer, "KD ", 3) == 0)
{
    float val = 0.0f;
    if(sscanf(rxBuffer + 3, "%f", &val) == 1)
    {
        Speed_PID.Kd = val;
        sprintf(msg, "\r\nKd set to %.3f\r\n", Speed_PID.Kd);
        HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
    }
}
// --- Lệnh không xác định ---
else
{
    sprintf(msg, "\r\nUnknown Command: %s\r\n", rxBuffer);
    HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
}
}

/*
=====
=====
* UART CLI RECEIVER & CALLBACKS
*
=====
===== */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART1)
    {
        // 1. Echo ký tự vừa gõ
        HAL_UART_Transmit(&huart1, (uint8_t*)&rxChar, 1, 10);

        // 2. Kiểm tra ký tự kết thúc dòng
    }
}

```

```

    if (rxChar == '\r' || rxChar == '\n')
    {
        if (rxIndex > 0)
        {
            rxBuffer[rxIndex] = '\0';
            cmdReady = 1;
            rxIndex = 0;
        }
    }
    else
    {
        if (rxIndex < (sizeof(rxBuffer) - 1))
        {
            rxBuffer[rxIndex++] = rxChar;
        }
        else
        {
            rxIndex = 0; // Chống tràn bộ đệm
        }
    }
}

// 3. Tiếp tục lắng nghe ký tự tiếp theo
HAL_UART_Receive_IT(&huart1, (uint8_t*)&rxChar, 1);
}
}

void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART1)
    {
        __HAL_UART_CLEAR_OREFLAG(huart);
        __HAL_UART_CLEAR_NEFLAG(huart);
        __HAL_UART_CLEAR_FEFLAG(huart);

        HAL_UART_Receive_IT(&huart1, (uint8_t*)&rxChar, 1);
    }
}

void MPU6050_GyroCalibration(void)
{

```

```
float sum = 0.0f;
```

```
for(int i = 0; i < 1000; i++)
```

```
{
```

```
    HAL_I2C_Mem_Read(&hi2c1,  
                      MPU6050_ADDR,  
                      0x43,  
                      I2C_MEMADD_SIZE_8BIT,  
                      &mpuData[8],  
                      6,  
                      HAL_MAX_DELAY);
```

```
    GyroX = (int16_t)(mpuData[8] << 8 | mpuData[9]);
```

```
    Gx = (float)GyroX / 131.0f;
```

```
    sum += Gx;
```

```
    HAL_Delay(2);
```

```
}
```

```
GxOffset = sum / 1000.0f;
```

```
    sprintf((char*)txBuf,  
            "Gyro X Offset = %.3f\r\n",  
            GxOffset);
```

```
HAL_UART_Transmit(&huart1,  
                  txBuf,  
                  strlen((char*)txBuf),  
                  HAL_MAX_DELAY);  
}
```

```
/* USER CODE END 4 */
```

```
/**
```



```

* @brief This function is executed in case of error occurrence.
* @retval None
*/
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 *        where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```

1. Nguyên nhân gây ra Drift & Lỗi trong Code

🚩 **Lỗi 1: Sai số Thời gian lấy mẫu DT (Nguyên nhân lớn nhất làm Roll tăng/giảm dần)**

- Thực tế trong code: Trong main.c, bạn đang định nghĩa `#define DT 0.1f` (tương đương 100ms) và bạn tính bộ lọc trong hàm `while(1)` có chứa `HAL_Delay(100)`.
- Vấn đề:
 1. `HAL_Delay(100)` cộng với thời gian đọc I2C và truyền UART khiến chu kỳ thực tế bị lệch (ví dụ thành 105ms – 110ms) chứ không tròn 100ms.
 2. Chu kỳ lấy mẫu góc 100ms (10Hz) là quá chậm cho robot tự cân bằng. Robot 2 bánh cần vòng lặp lấy mẫu góc ít nhất 100Hz (10ms).

3. Lỗi tích phân Gyro $Gx \times DT$ sẽ tích lũy sai số rất nhanh nếu DT bị lệch với chu kỳ thực tế.

🔔 Lỗi 2: Hàm Calibration Gyro chạy quá sớm khi MPU6050 chưa ổn định

- Vấn đề: Trong main(), ngay sau khi ghi thành ghi 0x6B = 0 (Wake up MPU), bạn gọi ngay hàm MPU6050_GyroCalibration().
- Thực tế phần cứng: Khi vừa thoát khỏi chế độ Sleep, các cảm biến cơ khí MEMS bên trong MPU6050 chưa ổn định điện áp và nhiệt độ. Calibration lúc này sẽ tính ra GxOffset sai lệch đáng kể.

🔔 Lỗi 3: Chế độ Power Management 1 (Register 0x6B) chưa chọn Clock Source tốt

- Vấn đề: Bạn ghi 0x00 vào thanh ghi 0x6B. Giá trị 0x00 bắt MPU6050 dùng dao động nội 8MHz (Internal 8MHz oscillator). Dao động nội này rất dễ bị trôi theo nhiệt độ.
- Khắc phục: MPU6050 khuyến cáo nên set thanh ghi 0x6B thành 0x01 để dùng PLL với trục Gyro X làm clock reference (ổn định hơn rất nhiều).

2. Giải đáp chi tiết các nghi vấn của bạn (Mục 10)

A. Về Calibration:

1. Có cần delay sau khi wake MPU6050 không?
 - CÓ. Cần HAL_Delay(100) đến 200ms sau khi ghi thành ghi 0x6B để cảm biến kịp định hình nguồn và dao động.
2. Calibration 1000 mẫu có đủ không?
 - ĐỦ. 1000 mẫu với HAL_Delay(2) ngón tầm 2 giây là đủ tốt. Tuy nhiên, robot phải nằm yên tuyệt đối trong thời gian này.
3. Có cần calibrate GyroY / GyroZ không?
 - NÊN. Dù chỉ dùng Roll (Gx), nhưng việc tính sẵn Offset cho GyroY, GyroZ rất đơn giản và giúp bạn không bị bối rối nếu sau này dùng đến.
4. Offset có bị sai do sensor chưa ổn định?
 - ĐÚNG. Đây chính là nguyên nhân dẫn đến Gx sau khi trừ offset vẫn bị sót lại một lượng nhỏ ($\approx 0.02 - 0.05^\circ/s$). Tích phân theo thời gian sẽ làm góc trôi từ từ.

B. Về Drift:

- Nếu $Gx \neq 0$ khi đứng yên: Là do Offset chưa chuẩn.
- Nếu $Gx \approx 0$ mà Roll vẫn tăng/giảm dần: Là do Bộ lọc Complementary chưa hoạt động đúng để kéo Roll về RollAcc, hoặc DT bị tính sai số.

3. Giải pháp khắc phục triệt để

Chúng ta sẽ chuyển toàn bộ quá trình đọc MPU6050 + tính Complementary Filter vào trong ngắt TIM4 (100Hz / 10ms) chung với vòng lặp điều khiển, loại bỏ việc đọc cảm biến trong while(1).

Bước 1: Sửa lại Macro DT và Khởi tạo MPU

Thay đổi trong #define:

C

```
/* USER CODE BEGIN PD */
#define ENCODER_CPR 1320.0f
#define SAMPLE_TIME 0.01f
#define RPM_FACTOR (60.0f / (ENCODER_CPR * SAMPLE_TIME))
#define MPU6050_ADDR (0x68 << 1)
#define M_PI 3.14159265358979323846
```

```
#define DT 0.01f // SỬA: DT = 10ms (100Hz) để khớp chính xác với ngắt TIM4
/* USER CODE END PD */
```

Bước 2: Nâng cấp hàm Calibration Gyro (có delay khởi động)

Sửa hàm MPU6050_GyroCalibration() trong main.c:

C

```
void MPU6050_GyroCalibration(void)
{
    float sum = 0.0f;

    // Đảm bảo robot nằm yên trong lúc này!
    for(int i = 0; i < 1000; i++)
    {
        HAL_I2C_Mem_Read(&hi2c1,
                        MPU6050_ADDR,
                        0x43,
                        I2C_MEMADD_SIZE_8BIT,
                        &mpuData[8],
                        2, // Chỉ cần đọc 2 byte GyroX
                        HAL_MAX_DELAY);

        int16_t raw_gx = (int16_t)(mpuData[8] << 8 | mpuData[9]);
        sum += ((float)raw_gx / 131.0f);

        HAL_Delay(2);
    }

    GxOffset = sum / 1000.0f;

    sprintf((char*)txBuf, "Gyro X Offset Calibrated: %.3f deg/s\r\n", GxOffset);
    HAL_UART_Transmit(&huart1, txBuf, strlen((char*)txBuf), HAL_MAX_DELAY);
}
```

Bước 3: Đưa việc tính Góc Roll vào Control_Loop() (Chạy chuẩn 100Hz trong TIM4)

Tạo hàm đọc và cập nhật góc MPU6050:

C

void MPU6050_Read_Angle(void)

```
{  
    // Đọc 14 bytes dữ liệu  
    if(HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR, 0x3B, I2C_MEMADD_SIZE_8BIT,  
mpuData, 14, 10) == HAL_OK)  
    {  
        AccY = (int16_t)(mpuData[2] << 8 | mpuData[3]);  
        AccZ = (int16_t)(mpuData[4] << 8 | mpuData[5]);  
        GyroX = (int16_t)(mpuData[8] << 8 | mpuData[9]);  
  
        Ay = (float)AccY / 16384.0f;  
        Az = (float)AccZ / 16384.0f;  
  
        // Trừ offset chính xác  
        Gx = ((float)GyroX / 131.0f) - GxOffset;  
  
        // Tính góc Roll từ Gia tốc  
        RollAcc = atan2f(Ay, Az) * 180.0f / M_PI;  
  
        // Complementary Filter với DT chuẩn = 0.01s (10ms)  
        // Formula: Roll = alpha * (Roll + Gx * DT) + (1 - alpha) * RollAcc  
        Roll = 0.98f * (Roll + Gx * DT) + 0.02f * RollAcc;  
    }  
}
```

Cập nhật lại Control_Loop() trong ngắt TIM4:

C

void Control_Loop(void)

```
{  
    // 1. Đọc góc nghiêng ở tần số 100Hz  
    MPU6050_Read_Angle();  
  
    // 2. Cập nhật Encoder  
    Encoder_Update();  
  
    // 3. Xử lý Chế độ điều khiển  
    if (control_mode == MODE_PID_SPEED)  
    {  
        Speed_PID.feedback = encoder_rpm;  
    }  
}
```

```

    Speed_PID.setpoint = target_rpm;
    float out = PID_Update(&Speed_PID);
    Motor_SetPWM((int)out);
}
else
{
    Motor_SetPWM(manual_pwm);
}
}

```

Bước 4: Sửa luồng khởi tạo trong main()

Trong hàm main(), sửa đoạn Wake up MPU và gọi Calibration:

C

```

/* USER CODE BEGIN 2 */
// ... (Giữ nguyên phần PWM, Encoder, PID_Init, Timer4)

// Wake up MPU6050 & Chọn Clock Source = PLL GyroX (0x01)
data = 0x01;
HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, 0x6B, I2C_MEMADD_SIZE_8BIT,
&data, 1, HAL_MAX_DELAY);

// Chờ MPU6050 ổn định điện áp và phần cơ MEMS
HAL_Delay(200);

// Calibration Gyro khi robot đứng yên
MPU6050_GyroCalibration();

// Khởi tạo giá trị ban đầu cho Roll bằng đúng RollAcc để tránh giật góc ban đầu
HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR, 0x3B, I2C_MEMADD_SIZE_8BIT,
mpuData, 6, HAL_MAX_DELAY);
AccY = (int16_t)(mpuData[2] << 8 | mpuData[3]);
AccZ = (int16_t)(mpuData[4] << 8 | mpuData[5]);
Roll = atan2f((float)AccY / 16384.0f, (float)AccZ / 16384.0f) * 180.0f / M_PI;

// Bắt đầu Timer ngắt chu kỳ 10ms (100Hz)
HAL_TIM_Base_Start_IT(&htim4);
/* USER CODE END 2 */

```

Và làm sạch vòng lặp while(1) (loại bỏ việc đọc MPU trùng lặp trong while):

C

```

/* USER CODE BEGIN WHILE */
while (1)

```

```

{
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
  if (cmdReady)
  {
    cmdReady = 0;
    Process_Command();
  }

  // In dữ liệu UART ra ngoài màn hình MONITOR / PLOTTER
  Process_Print();
}
/* USER CODE END 3 */

```

4. Cách kiểm tra dữ liệu qua UART (Mục 11)

Sau khi nạp code đã cập nhật ở trên, hãy bật Terminal UART để quan sát chuỗi in ra:

1. Kiểm tra Gyro X Offset: Lúc khởi động, xem Terminal báo Gyro X Offset Calibrated: Giá trị này thường nằm trong khoảng -3.0 đến +3.0 deg/s.
2. Kiểm tra khi Robot đặt cố định trên mặt phẳng:
 - Gx phải dao động quanh $0.00 \pm 0.05^\circ/s$.
 - RollAcc đứng yên một giá trị (ví dụ 1.2°).
 - Roll phải ổn định tuyệt đối quanh giá trị của RollAcc, không được trôi tăng/giảm theo thời gian.
3. Kiểm tra độ nhạy động: Nghiêng robot sang trái/phải rồi giữ nguyên ở góc mới, quan sát giá trị Roll xem có bám kịp theo RollAcc nhanh chóng và không bị đung đưa không.

Thử nghiệm xong phần này, nếu góc Roll đã phẳng và ổn định, bạn đã sẵn sàng 100% để bước sang cài đặt Angle PID!

- Vấn đề nghiêm trọng, MPU drift
- thử debug kiểu mới -> giải quyết vấn đề bị gián đoạn khi đang quay -> cân nhắc không sử dụng UART quá nhiều gây gián đoạn hoặc tìm một cách nào đó để giữ cho việc truyền tín hiệu trở nên đều đặn và có chu kì hơn

Angle_PID.output	0.000	float
Angle_PID.Kp	457	float
Angle_PID.Ki	1.29999995	float
Angle_PID.Kd	4.5999999	float
BalanceAngle	-0.330000013	float
<Enter expression>		

Thông số này cân bằng biên độ nhỏ ổn định vô thời hạn

Một số vấn đề cần cải thiện ở "Phần cứng":

- thiết kế sao cho có thể điều chỉnh được trọng tâm của robot
- Độ tin cậy của một số chi tiết cần được đảm bảo với tiêu chuẩn cao
- + khe đựng mạch có chứa MPU phải cố định, không được lung lay hoặc dễ bị điều chỉnh
- + độ rơ của động cơ
- + một số chi tiết hỗ trợ tuning cho robot: cái móc trên đầu...
- + bảo vệ mạch (các chân mạch nạp ở stm32 dễ bị gãy khi debug)